Training

exp022 · 28 June 2026

Abstract

This is the gamma-gated-sparsity collection's training hub: 87 networks trained once to a shared root that every downstream analysis loads instead of retraining (train-once / reuse-many). We trained all 87 to the gamma operating point in a single fan-out run, three seeds each. The sweeps carry the collection's headline results: under a tightening per-neuron spike budget PING degrades gracefully while COBA collapses, a gap widening to ≈ 26 accuracy points; the loop's gamma frequency tracks the inhibitory time constant across the τ_GABA ladder; and only the built-in E/I loop keeps its rhythm when the recurrent weights are left free to train.

Methods

1. The compute cost

Each trial is surrogate-gradient backprop-through-time: 2000 timesteps ($T = 200$ ms at $\Delta t = 0.1$ ms) over a 1280-neuron conductance network. What makes it costly:

- **Fine timestep.** COBA synapses inject $g(V - E)$, stiffening the membrane so $\Delta t = 0.1$ ms, 10× finer than a current-based model, and non-optional: the gamma rhythm lives there.
- **Sequential.** The 2000 steps are a dependency chain no hardware parallelises, so the job is **bandwidth-bound**: a GPU beats a CPU only ≈ 10×, not the 50–100× of large matmuls.
- **Memory-heavy.** The backward pass stores every step's activations, ≈ 12 GB at batch 256.
- **Scale.** 87 cells × 50 epochs ≈ **185 A100-GPU-hours** (≈ 49M sample-forwards), ≈ 159 days on CPU, so every cell trains on a GPU.

2. Gold star trainings

87 cells across five families, each fixing its time constants, timestep, and MNIST fraction (re-split 80/20). The standard is $\tau_{\text{AMPA}} = 2$ ms, $\tau_{\text{GABA}} = 6$ ms (loop in gamma, ≈ 44 Hz); the sweeps each vary one axis: spike budget, τ_GABA, timestep (exp044), or init.

Family	τ_{GABA} (ms)	Δt (ms)	Epochs	MNIST	Cells	Trained
canonical (no budget)	6	0.1	50	all (70k)	6	6 / 6
spike-budget sweep	6	0.1	50	10%	36	36 / 36
τ_GABA ladder	4.5 – 27	0.1	50	10%	18	18 / 18
Δt sweep	6	0.05 – 1.0	50	10%	15	15 / 15
init variants	6	0.1	50	10%	12	12 / 12
total					**87**	**87 / 87**

The **canonical** family (coba, ping with no spike budget) is the full-MNIST reference the sweeps are read against. Every cell was retrained fresh to the gamma standard, replacing the older notebook cells (which had run at 2.9–5% MNIST and $\tau_{\mathrm{GABA}} = 9$ ms), so all 87 are drawn at one consistent operating point.

Choices behind the table:

- **50 epochs, halved from 100.** Accuracy plateaus by ≈ 15–20 epochs (exp024); 50 keeps a ≈ 30-epoch tail for the post-convergence *dynamics* the collection studies (rate drift, confidence inflation) while nearly halving the run.
- **3 seeds throughout.** Every cell, canonical and every sweep, trains three seeds (42, 43, 44), so each point on each frontier carries a cross-seed band and the headline effects (PING's rate attractor) read as robust, not one-run luck. The spike-budget interior used to be single-seed; it no longer is.
- **Full MNIST vs 10%.** The canonical reference carries the headline numbers, so it sees all 70k; the sweeps need only the trend across their parameter, so 10% suffices and cuts their cost tenfold.

3. Compute options

Bandwidth is the main driver but not the whole story: the RTX 4090 below has half the A100's bandwidth yet measured *faster*. Costs and wall-clocks cover the full 50-epoch registry; measured where the card was to hand, projected for the 5090:

Option	GPU (bandwidth)	Samples/s	Full-run cost	Wall-clock
Modal	A100 (2.0 TB/s)	74	≈ $615	≈ half a day (fans out)
RunPod / Vast.ai	RTX 4090 fleet (1.0 TB/s)	100 (meas.)	≈ $47 – 95	≈ 10 hr on ≈ 15 pods (≈ 5.7 d serial)
Cambridge Wilkes3	A100	74	≈ £102	offline through 2026
benjy (CUED, shared)	A6000 (768 GB/s)	26.5	£0	≈ 22 days
Owned workstation	RTX 5090 (1.8 TB/s)	≈ 150 (proj.)	≈ £4,700 once	≈ 3.8 days

The 4090 (≈ 100 samples/s, measured) beats both its bandwidth projection (≈ 37) and the A100, so the 5090 is scaled from it (≈ 1.5×). Wilkes3 is cheapest-and-fast but offline through 2026; benjy is one shared GPU forcing an older PyTorch.

How we ran it

The run fanned out across RunPod **RTX 4090s** in one datacenter (EU-RO-1), split by stakes: the 6 heavy canonical cells (≈ 9.7 hr each) on **secure on-demand** pods, one cell each; the 81 light sweep cells packed onto cheaper **community**-priced pods. A pre-baked cu128 Docker image let each pod boot ready to train, check out a pinned commit, train its cells to a shared network volume, and self-terminate, so the laptop was needed only to fire the fleet and collect the results afterwards. All 87 cells trained cleanly in ≈ 8 hours of wall-

clock for ≈ $70, against Modal's ≈ $615; the wall-clock is floored by a single canonical cell, which cannot be split, so more pods would not help. The three $\Delta t = 0.05$ ms cells needed ≈ 31 GB and finished on a 5090 rather than the 24 GB 4090. The owned **5090** remains the long-term answer for routine iteration.

Results

1. Canonical reference (no spike budget, all MNIST)
coba and ping with no spike budget, all 70k images, seeds 42/43/44 (6 cells), the full-data baseline. Unconstrained, the two architectures are essentially tied: coba reaches ≈ 95.5 % and ping ≈ 94.0 %, with near-zero across-seed spread. This is the reference point from which the spike-budget sweep pulls them apart.

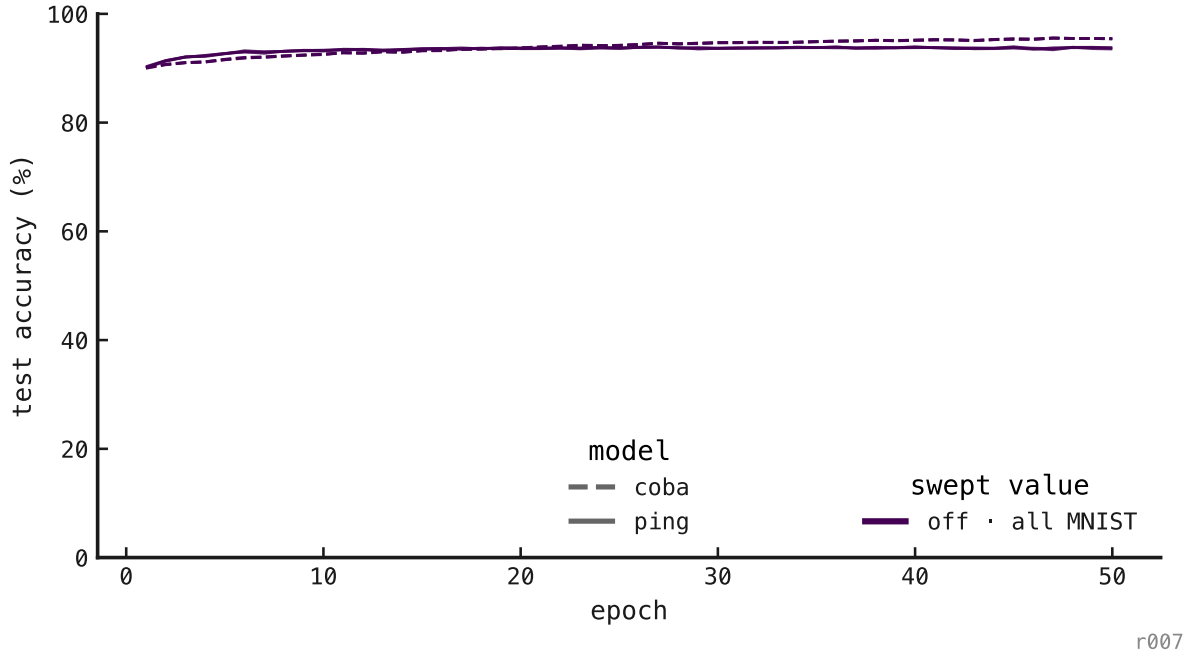

Figure 1: Test accuracy over epochs, canonical full-MNIST cells (coba dashed, ping solid; three seeds each). The two loops reach parity when no spike budget is imposed.

2. Spike-budget sweep
coba and ping across spike budgets ∈ off, 5, 2, 1, 0.5, 0.2, three seeds each (36 cells), so the accuracy–rate frontier carries error bars at every point. The spike budget is a per-neuron cap on firing (spikes/trial); lower is tighter. This is the headline result: as the budget tightens, ping degrades gracefully (91.6 → 86.5 %) while coba collapses (90.7 → 60.1 %), a gap that widens monotonically from ≈ 0 to ≈ 26 points. ping's γ-rhythm gates sparsity; coba's bare feedforward code cannot pay the budget without shedding accuracy.

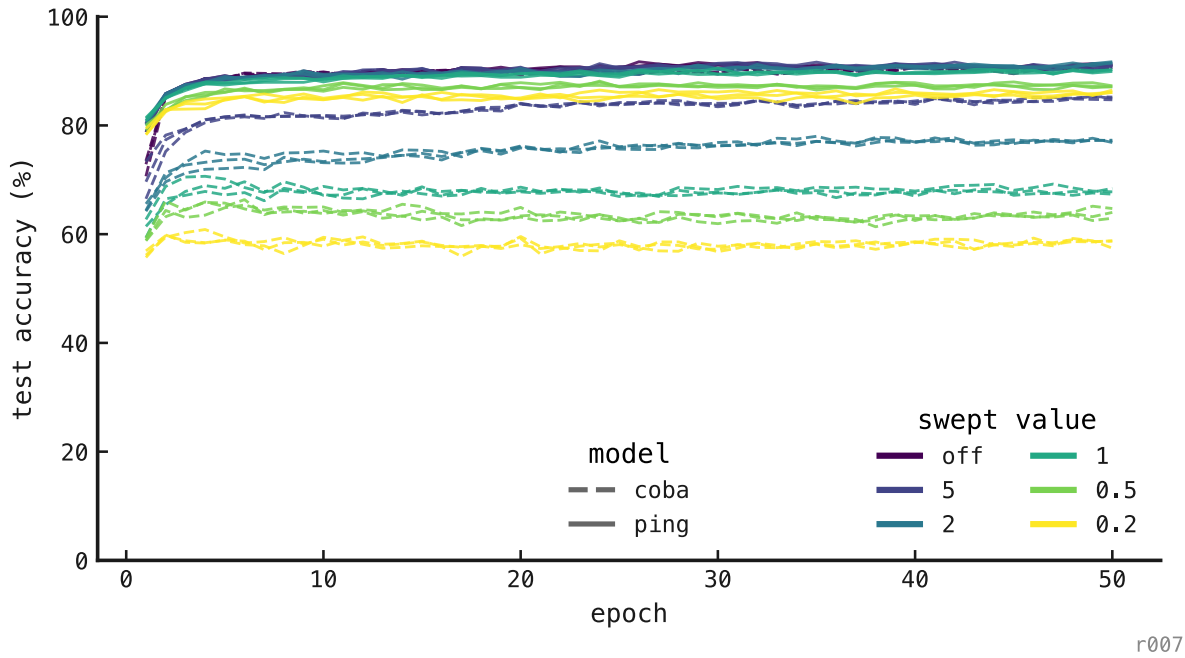

Figure 2: Test accuracy over epochs across the spike-budget sweep. Tighter budgets plateau lower (the spike-economy trade-off), and coba falls far faster than ping.

Scope. This frontier is a 10%-MNIST result: the whole sweep trains on the subset (§2 of Methods), and we have not re-run it at full data. The comparison is nonetheless internally clean: every cell here shares the same data fraction, so the coba-vs-ping gap is a budget effect, not a data-fraction artifact. And the direction of the density difference (§6) makes 10% the **conservative** regime for this claim: at full MNIST the no-budget baseline fires ≈ 2× harder, so a tight per-neuron cap would have further to pull coba down, widening the gap rather than closing it. We therefore read the ≈ 26-point separation as a floor, and do not claim the absolute numbers transfer unchanged to full MNIST.

3. τ_GABA ladder

ping across τ_GABA ∈ 4.5, 6, 9, 12, 18, 27 ms, three seeds each (18 cells). Accuracy is largely insensitive to inhibitory decay (≈ 88–92 % across the ladder), but the **rhythm** is not: measured from the trained networks, the γ-frequency falls monotonically with τ_GABA (≈ 50 Hz at 4.5 ms → ≈ 19 Hz at 27 ms), sitting at ≈ 45 Hz at the canonical τ_GABA = 6 ms, matching the operating point (Appendix).

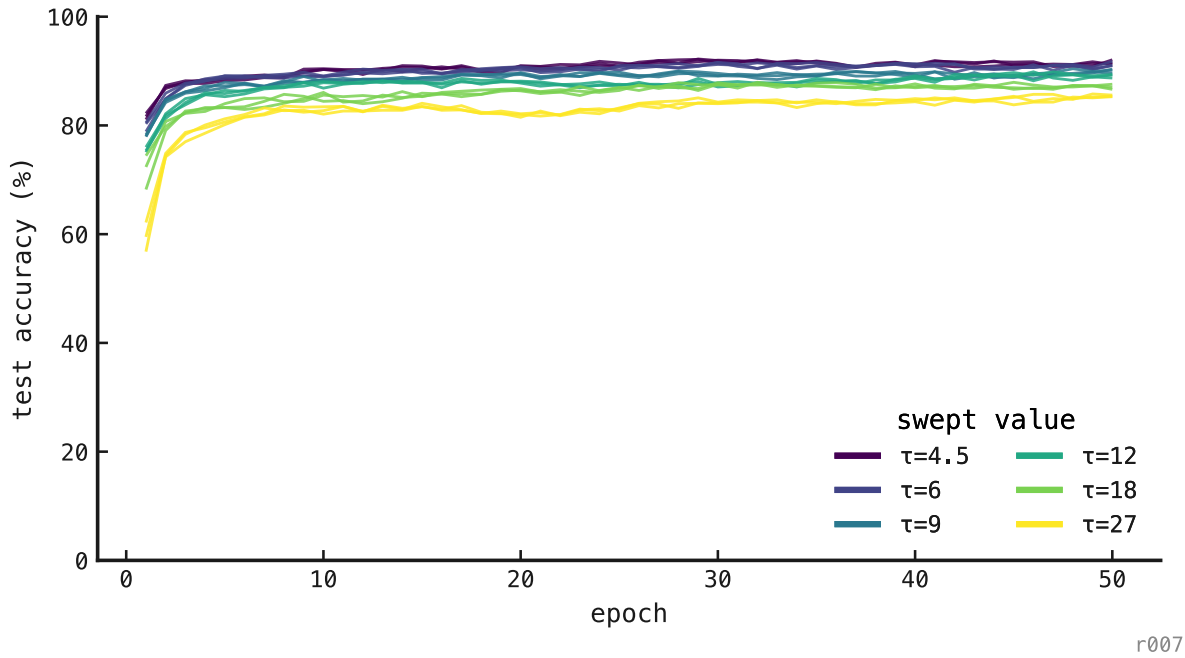

Figure 3: Test accuracy over epochs across the τ_GABA ladder; cells converge to similar accuracy regardless of inhibitory decay.

4. Δt sweep

ping across Δt ∈ 0.05, 0.1, 0.25, 0.5, 1.0 ms (physical T fixed), three seeds each (15 cells), the documented timestep exception. Accuracy is flat across the sweep (≈ 90.4–91.4 %): the integrator is robust to timestep from 0.1 to 1.0 ms, and the 0.05 ms cells (which need ≈ 31 GB and so ran on a 5090) agree.

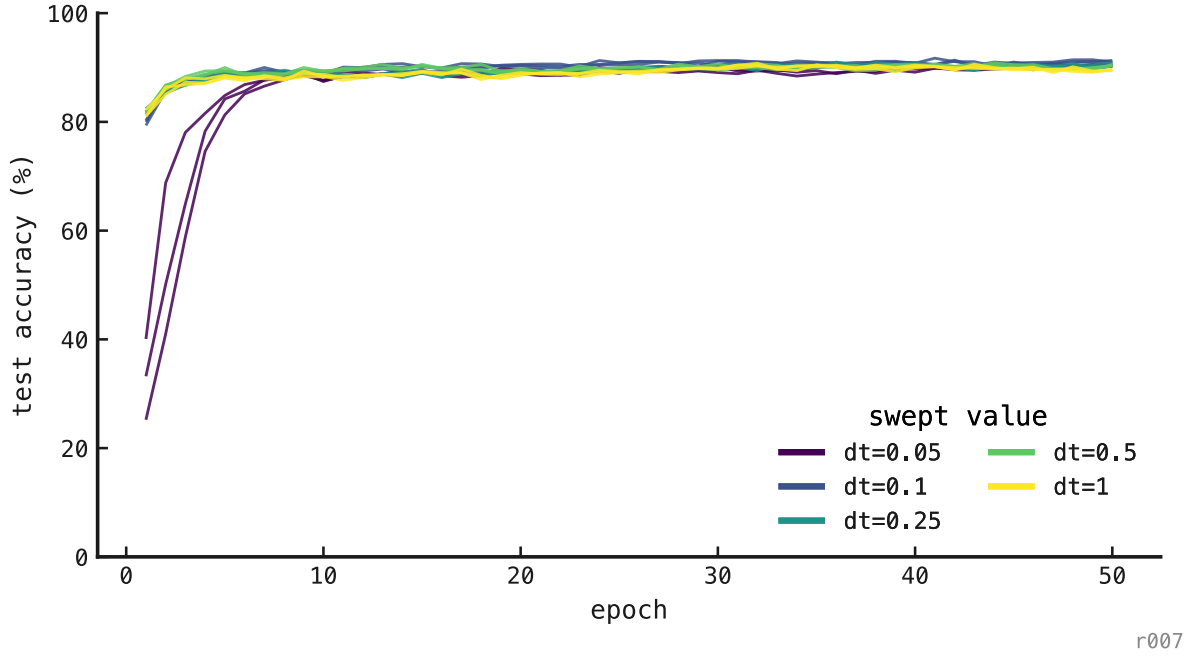

Figure 4: Test accuracy over epochs across the integration-timestep sweep; accuracy is insensitive to Δt over the tested range.

5. Init variants

ping with four recurrent-loop inits (frozen PING, trainable from PING / zero / small seed), three seeds each (12 cells). All reach ≈ 89–91 %, but only the frozen-PING control keeps the true E/I regime (E ≈ 10 Hz, I ≈ 62 Hz): the trainable-loop cells drift toward a feedforward code (high E, low or zero I): the zero-init cells never engage inhibition at all (I ≈ 0 Hz). Comparable accuracy, but the rhythm is not learned when it is not built in.

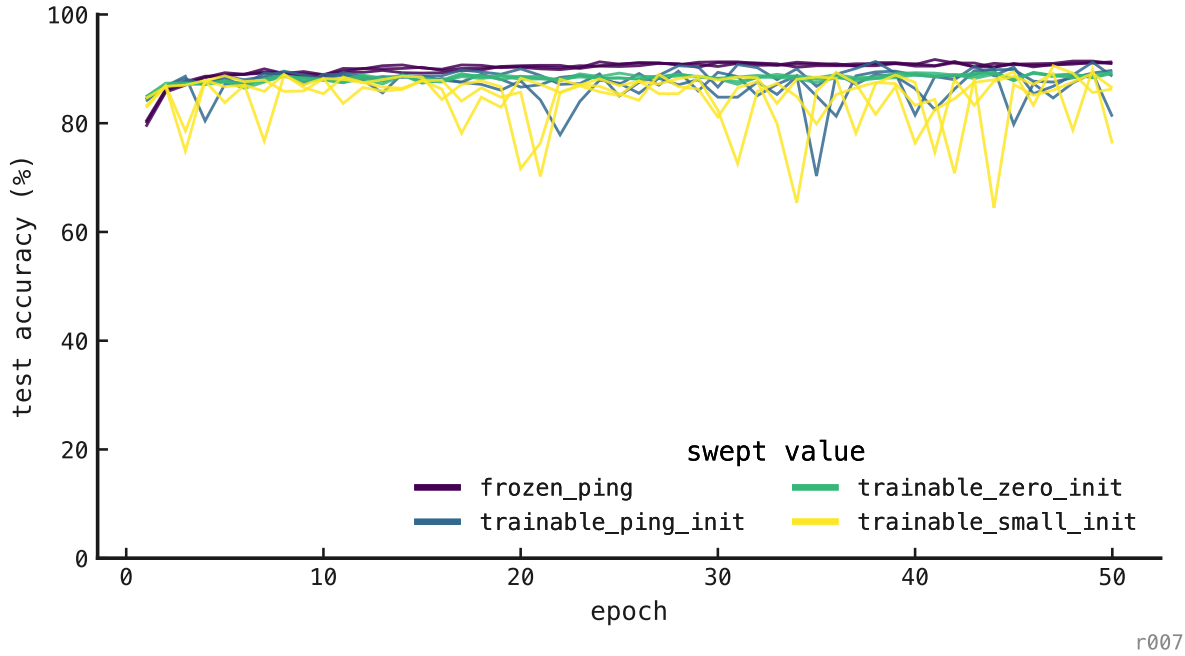

Figure 5: Test accuracy over epochs across the recurrent-loop inits; trainable-loop cells learn noisier curves than the frozen control.

6. Training data and spike density

The appendix rasters split cleanly along one axis that is easy to miss: the **canonical** cells see all 70k MNIST images, but every sweep (including the no-budget spike-budget = off cell, which is otherwise identical to canonical) trains on 10%. That difference deserves its own read, because it changes how **busy** the trained network is before any sweep parameter enters.

We isolate it by sending the **same** fixed digit-0 image through the no-budget coba and ping networks at each fraction (seed 42). Same architecture, same operating point ($\tau_{\text{AMPA}} = 2$ ms, $\tau_{\text{GABA}} = 6$ ms), same spike budget (none): the only difference is 10× the training images, so any gap in the rasters is the data's doing alone.

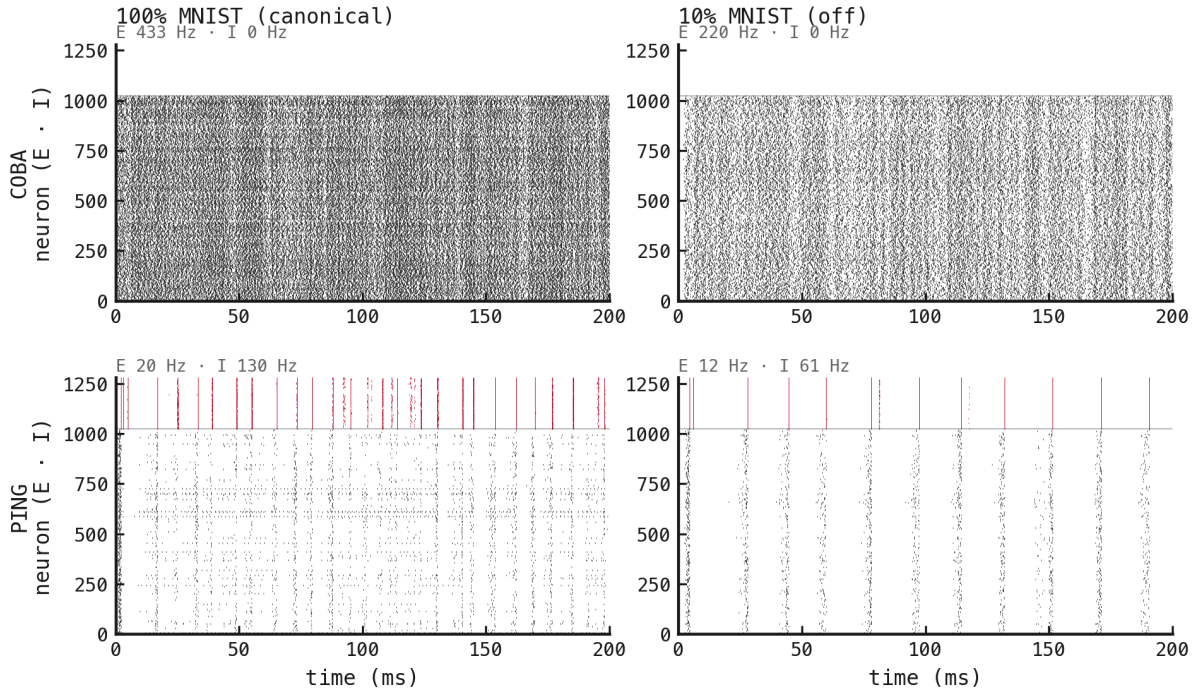

Figure 6: The same digit-0 image through the no-budget coba (top) and ping (bottom) networks, trained on all of MNIST (left) versus 10% (right); E cells black below the divider, I cells red above, per-panel mean rates annotated. More training data yields a visibly denser code in both architectures.

More data buys a denser code. In both loops the full-MNIST network fires roughly twice as hard:

- **coba** excitatory rate ≈ 420 Hz on all of MNIST against ≈ 212 Hz on 10%;
- **ping** inhibitory rate ≈ 140 Hz against ≈ 63 Hz (excitatory ≈ 19 vs ≈ 11 Hz).

where the **mean rate** is a population's total spikes divided by its neuron count and the 200 ms window. The canonical rasters are visibly busier: the extra spikes are the network recruiting capacity to separate the fuller set of class variations, which the 10% subset never forces it to. ping keeps its γ rhythm at both fractions, and the sparser 10% network even reads as a **cleaner** gamma (Appendix A.2); coba stays asynchronous throughout (I silent, no loop). The density shift is a change of degree, not regime.

The practical consequence is a caveat on the appendix as a whole: **absolute** firing rates are not comparable across the canonical-vs-sweep boundary, because the 10% cells sit at a systematically lower density for reasons that have nothing to do with the swept parameter. This is exactly why the canonical reference (§1) carries the headline rates and the sweeps are read as **trends within a family** (the spike-budget frontier, the τ_GABA-to-γ scaling) rather than as absolute numbers set against the full-data baseline.

7. Why more data drives a denser code

The figure measures the density gap; it does not prove its cause. But only two things change between a row's two cells (the number of training images and, downstream of that, the number of weight updates), and both push firing **up**. Neither is specific to the E/I loop,

which is why coba and ping move together rather than one architecture reacting and the other not.

- **More to separate.** The 70k-image set carries far more within-class variation than its 10% subset. To keep the ten digit classes separable at the readout (a linear map from per-neuron spike counts to class scores), the network must partition a higher-dimensional representation, and it pays for that resolution in spikes: more neurons recruited, each firing more. The smaller subset is an easier separation problem that a sparser code already fits, so the network never has to spend the extra spikes.
- **More weight updates.** Epochs are fixed at 50 for both, but an epoch over 70k images is $\approx 10\times$ the mini-batches of an epoch over 10%, so the canonical cells take $\approx 10\times$ as many gradient steps. These are the **no-budget** cells, so nothing opposes the drift: each update is free to grow the input and recurrent weights that set the drive $g(V - E)$, and more updates compound into a higher operating point.

where the terms are:

- g — the synaptic conductance a presynaptic spike opens (larger weights $\rightarrow$ larger g);
- V — the postsynaptic membrane voltage;
- E — the synapse's reversal potential, so $g(V - E)$ is the current a spike injects.

The spike budget is precisely the counter-pressure to both levers: the sweep's tightening θ_u (the per-neuron cap on spikes/trial) caps the rate growth described here, which is why the budgeted cells sit far below their no-budget siblings. The canonical–off pair simply removes that cap, leaving data fraction as the only lever. A clean way to separate the two mechanisms would be to train a 10% cell for $\approx 10\times$ the epochs, matching the update count at the smaller set: if the rate closes most of the gap the growth is update-driven, and if it does not the residual is the genuine cost of the harder separation. We have not run that control; the two levers are offered as the mechanism the rasters are consistent with, not as a decomposition we have measured.

Appendix — per-config spike rasters (digit 0)

The same fixed MNIST image (digit 0, sample 0) sent through each trained network (seed 42 as the per-config representative), so every raster is directly comparable. E cells sit below the divider, I cells above; the lower panel is the 1 ms population rate. These are the visual counterpart to the firing-rate and rhythm numbers above: sparse γ-rhythmic PING versus dense asynchronous COBA, and how each regime deforms under the sweeps.

A.1 Canonical reference

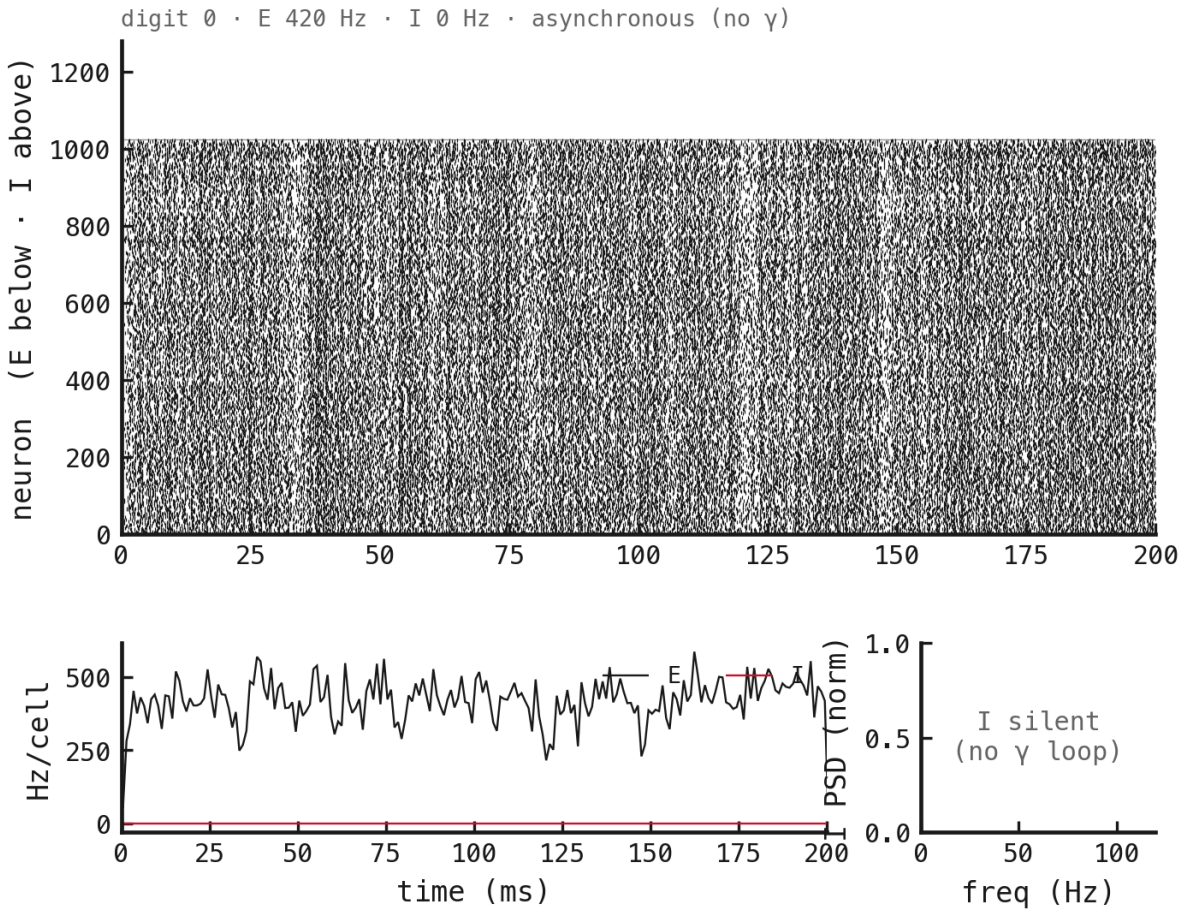

Figure 7: COBA, canonical (no spike budget, all MNIST).

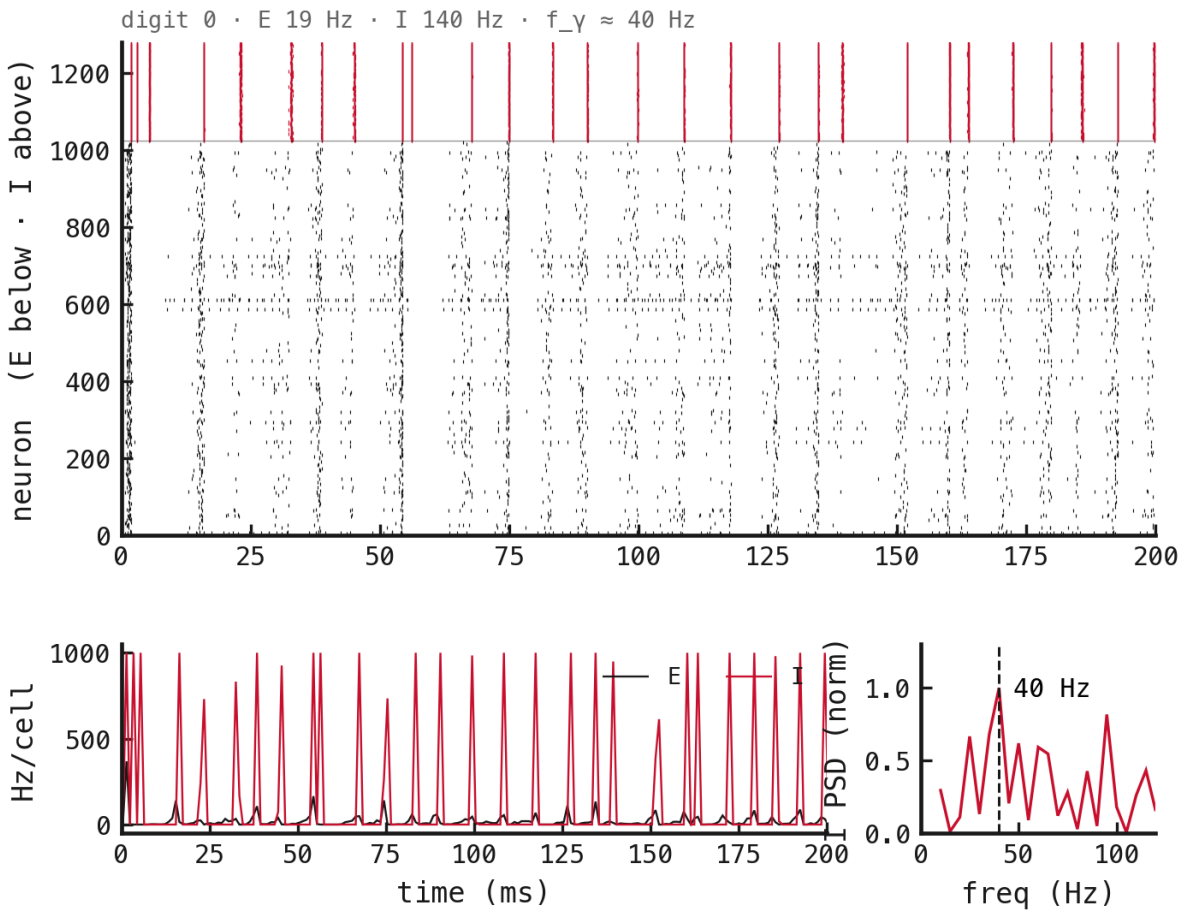

Figure 8: PING, canonical (no spike budget, all MNIST).

A.2 Spike-budget sweep

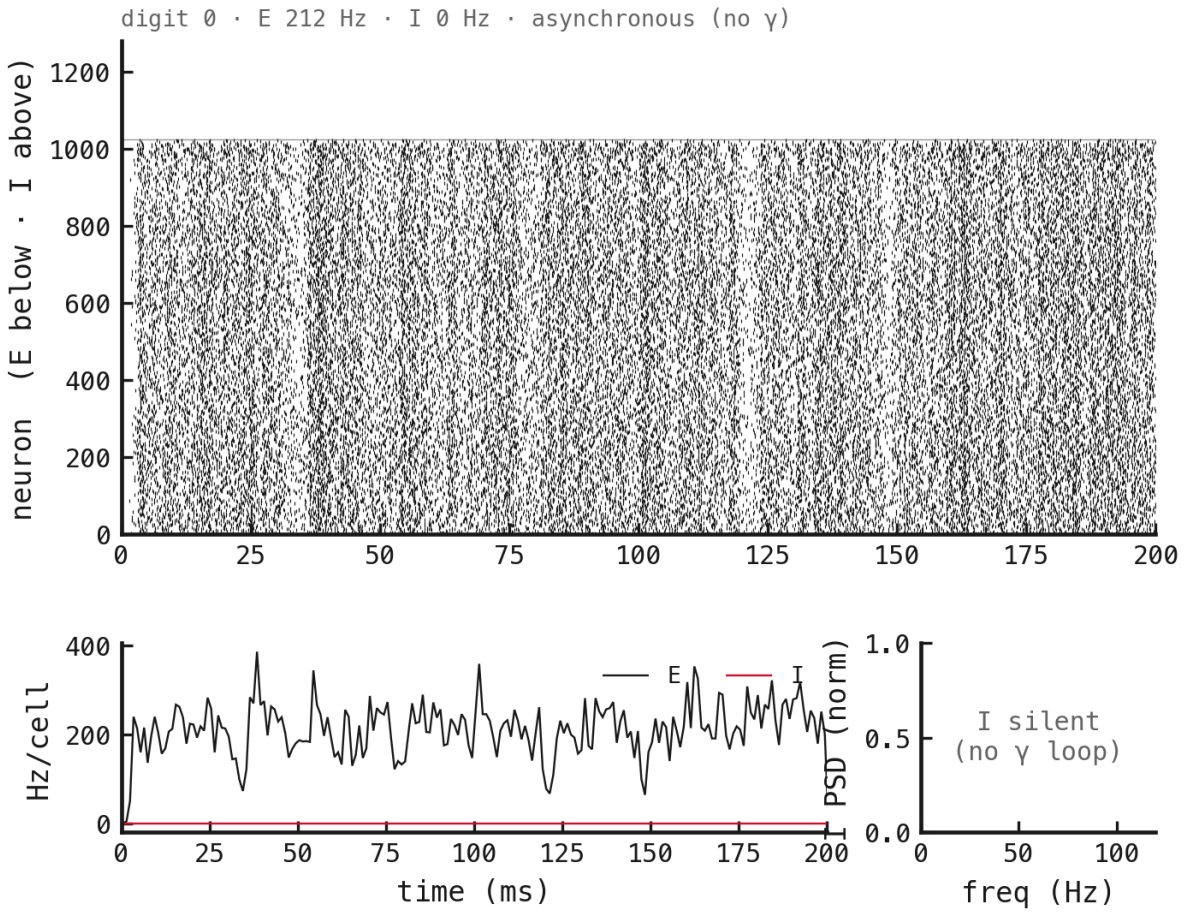

Figure 9: COBA, spike budget = off.

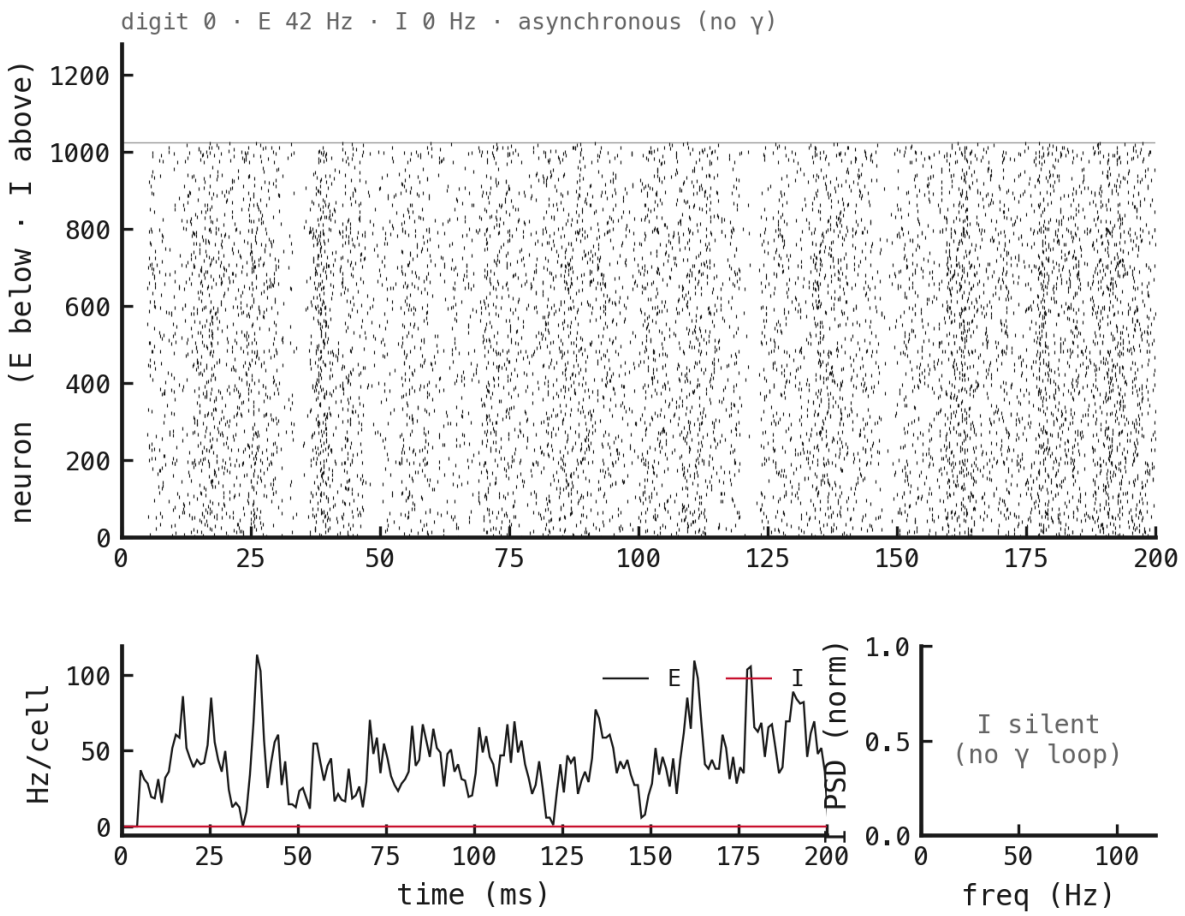

Figure 10: COBA, spike budget = 5.

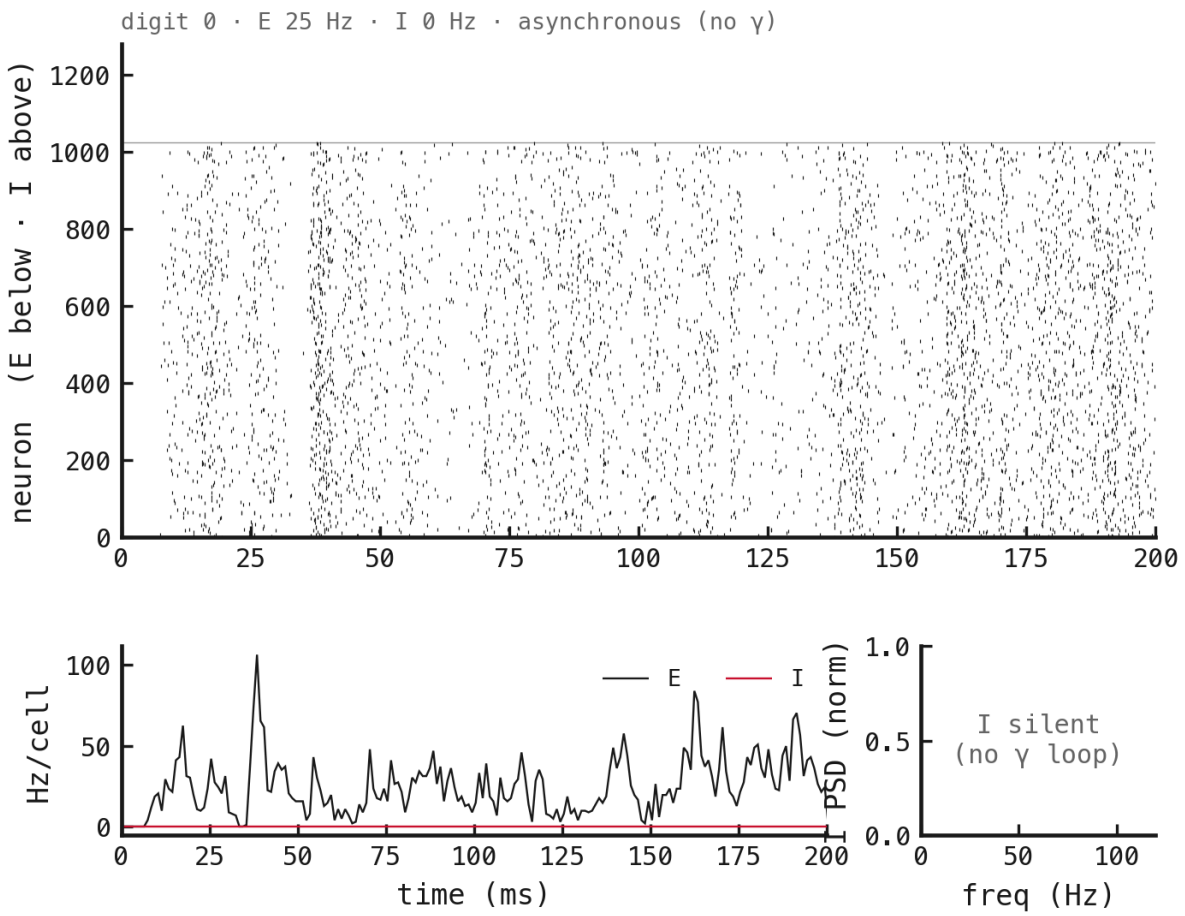

Figure 11: COBA, spike budget = 2.

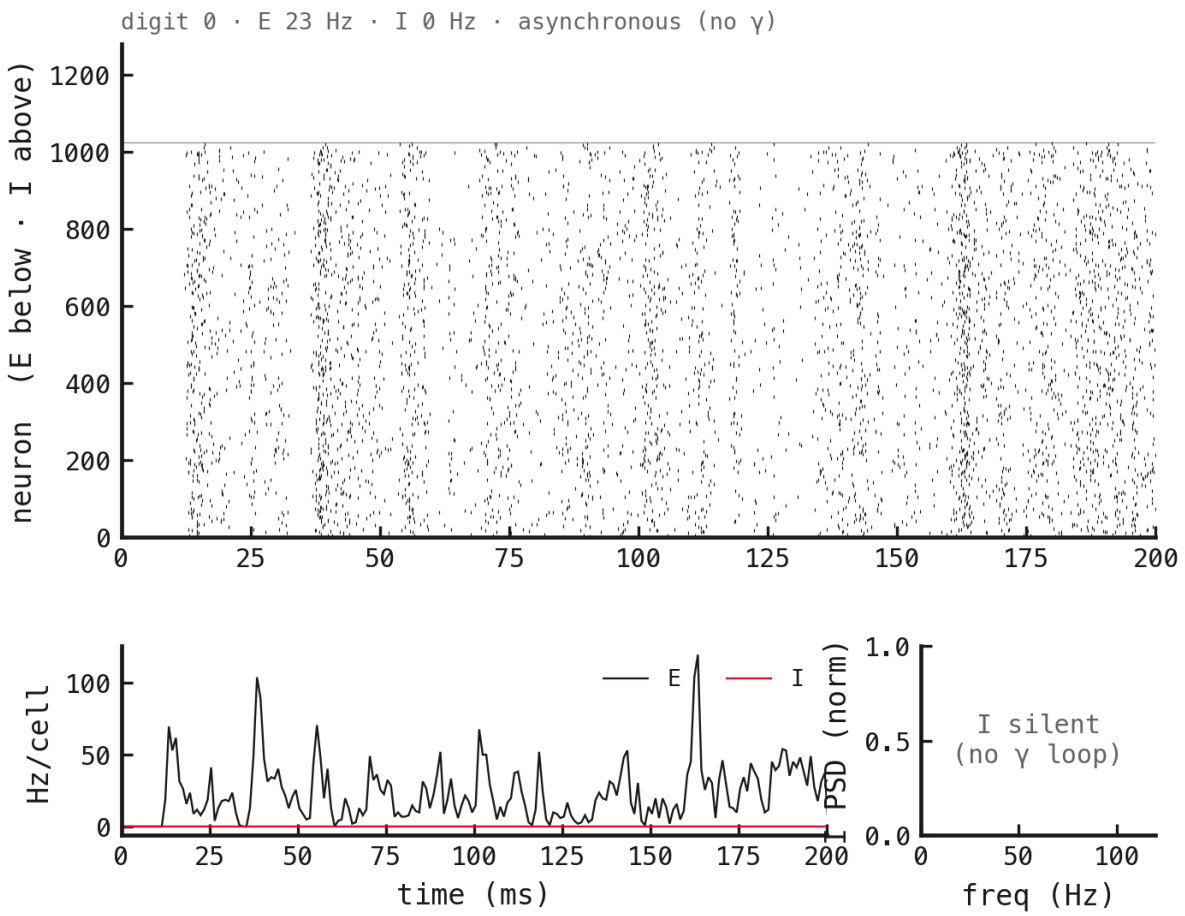

Figure 12: COBA, spike budget = 1.

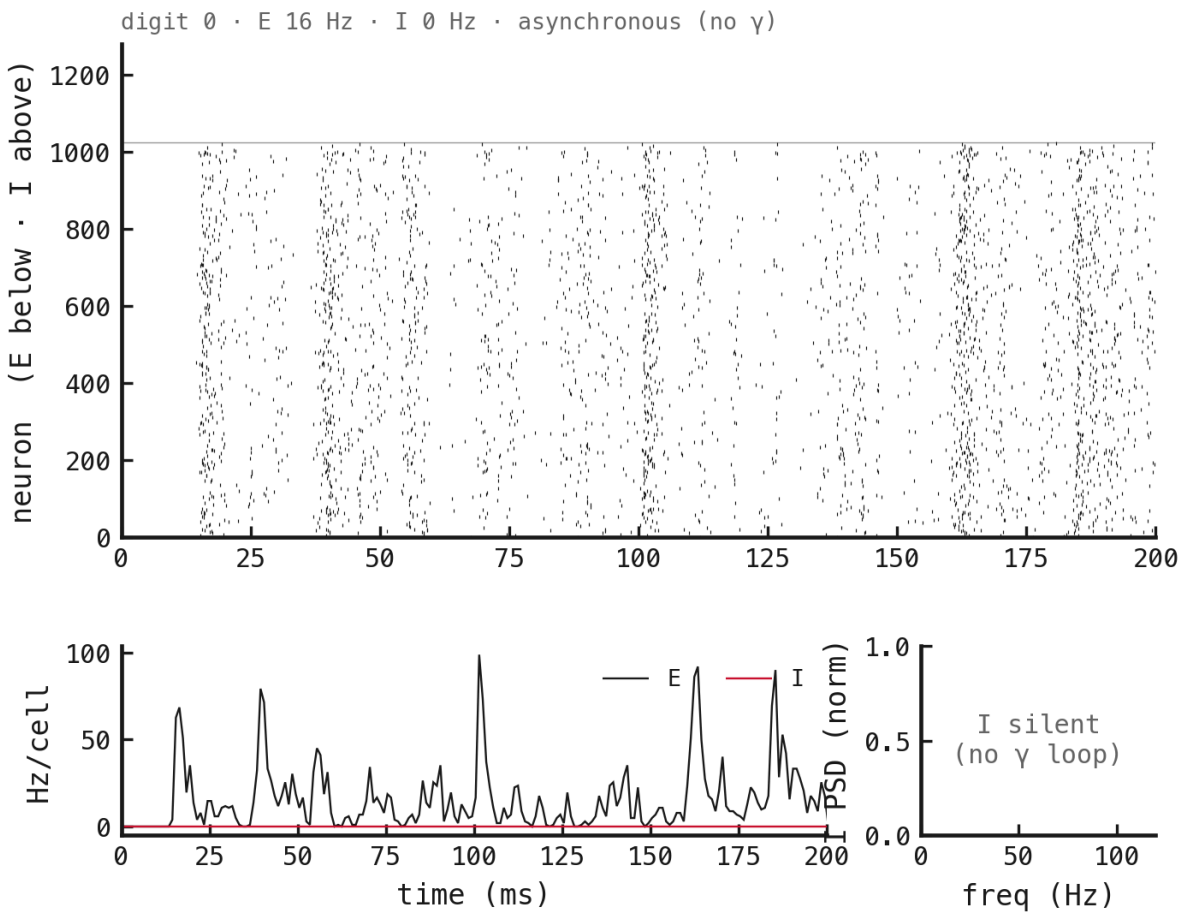

Figure 13: COBA, spike budget = 0.5.

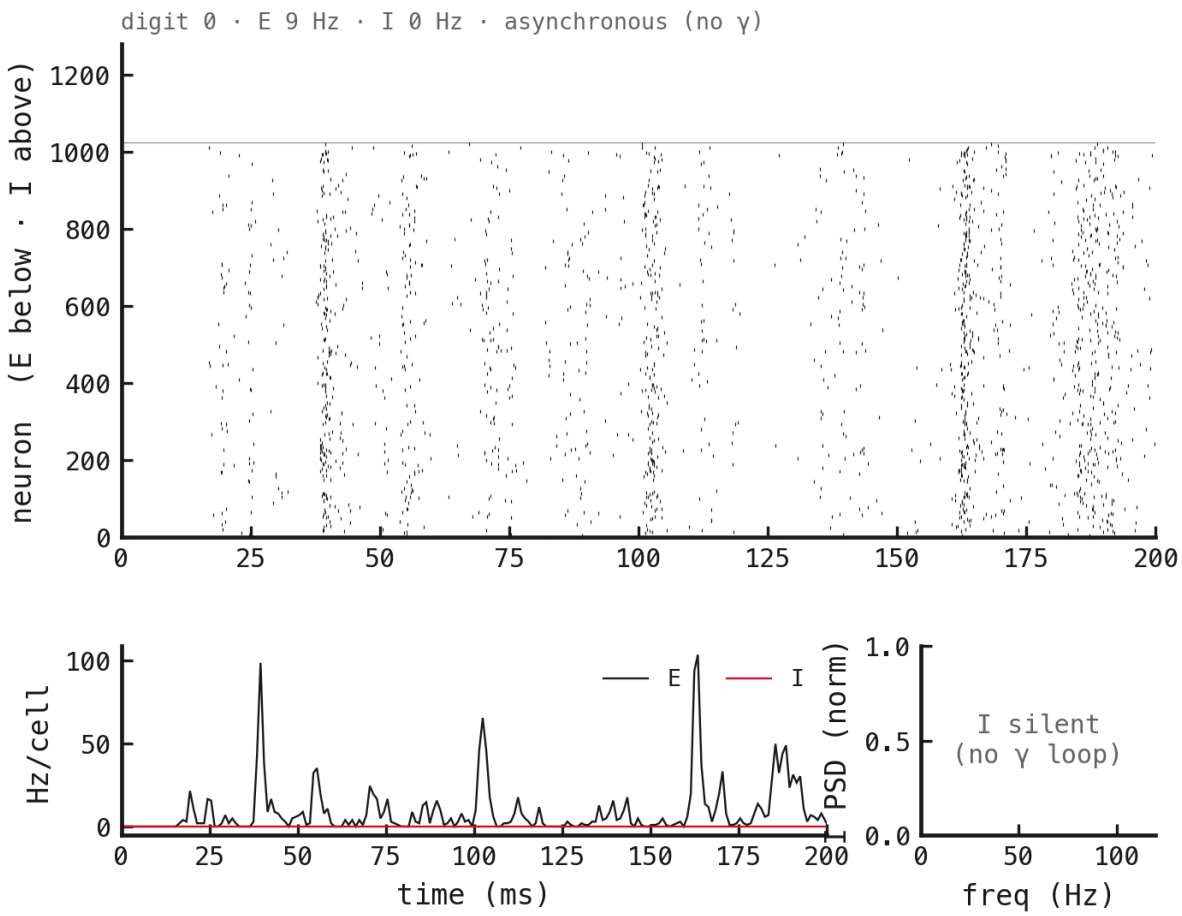

Figure 14: COBA, spike budget = 0.2.

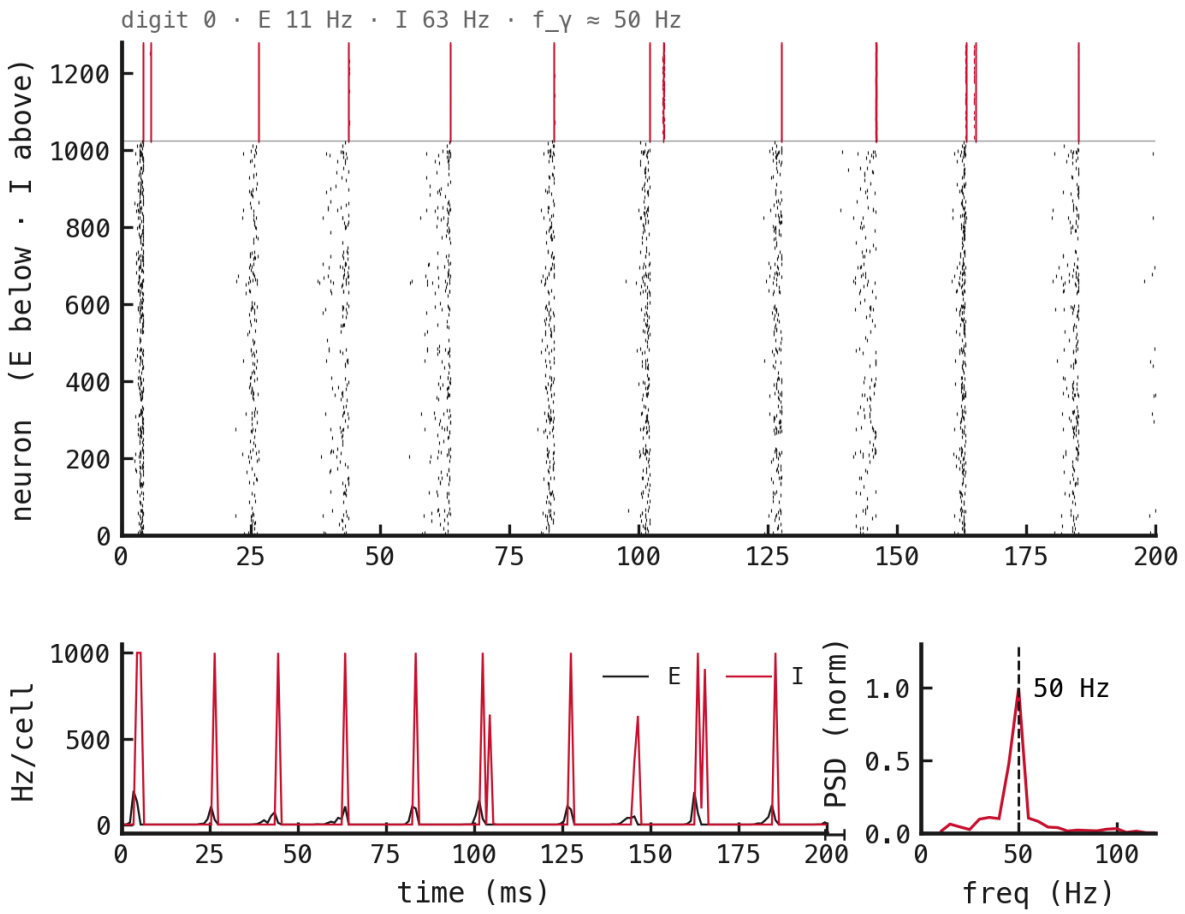

Figure 15: PING, spike budget = off.

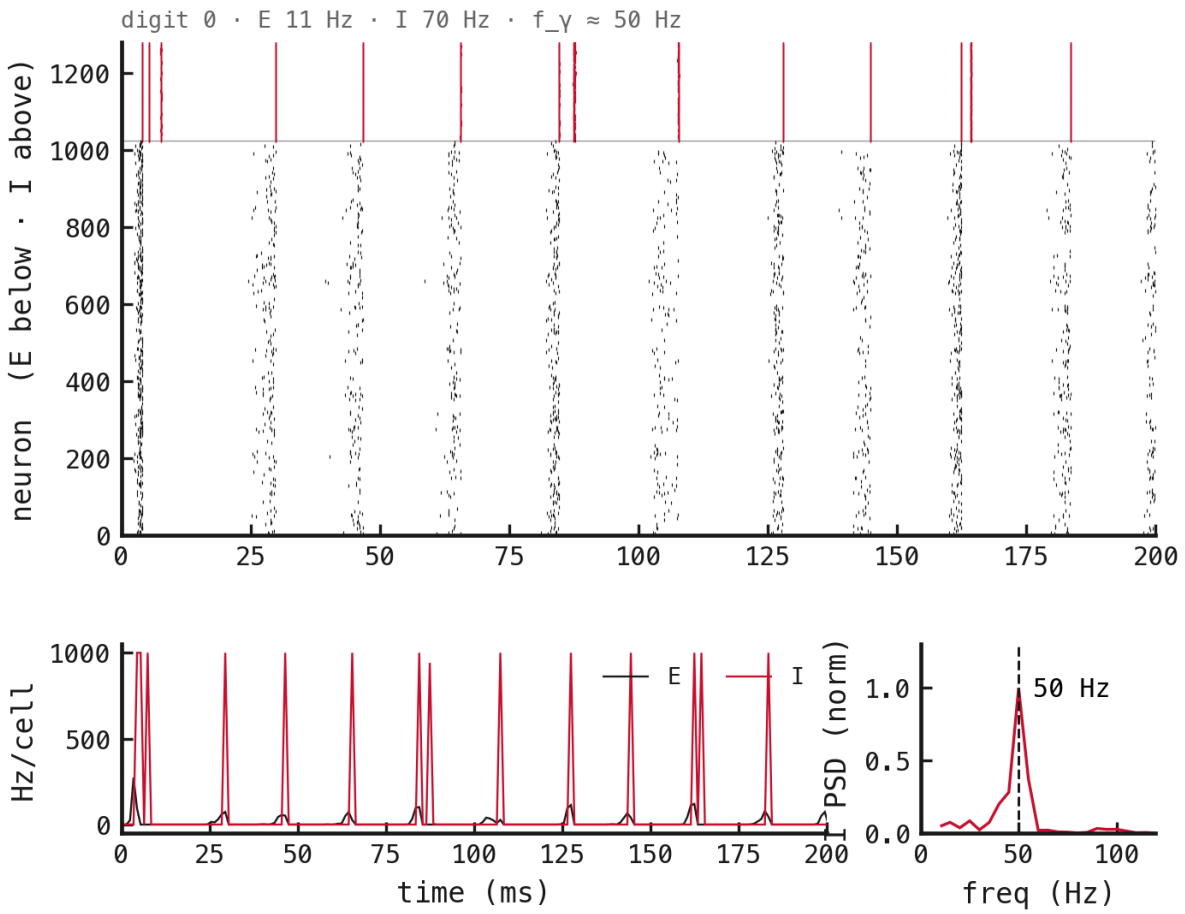

Figure 16: PING, spike budget = 5.

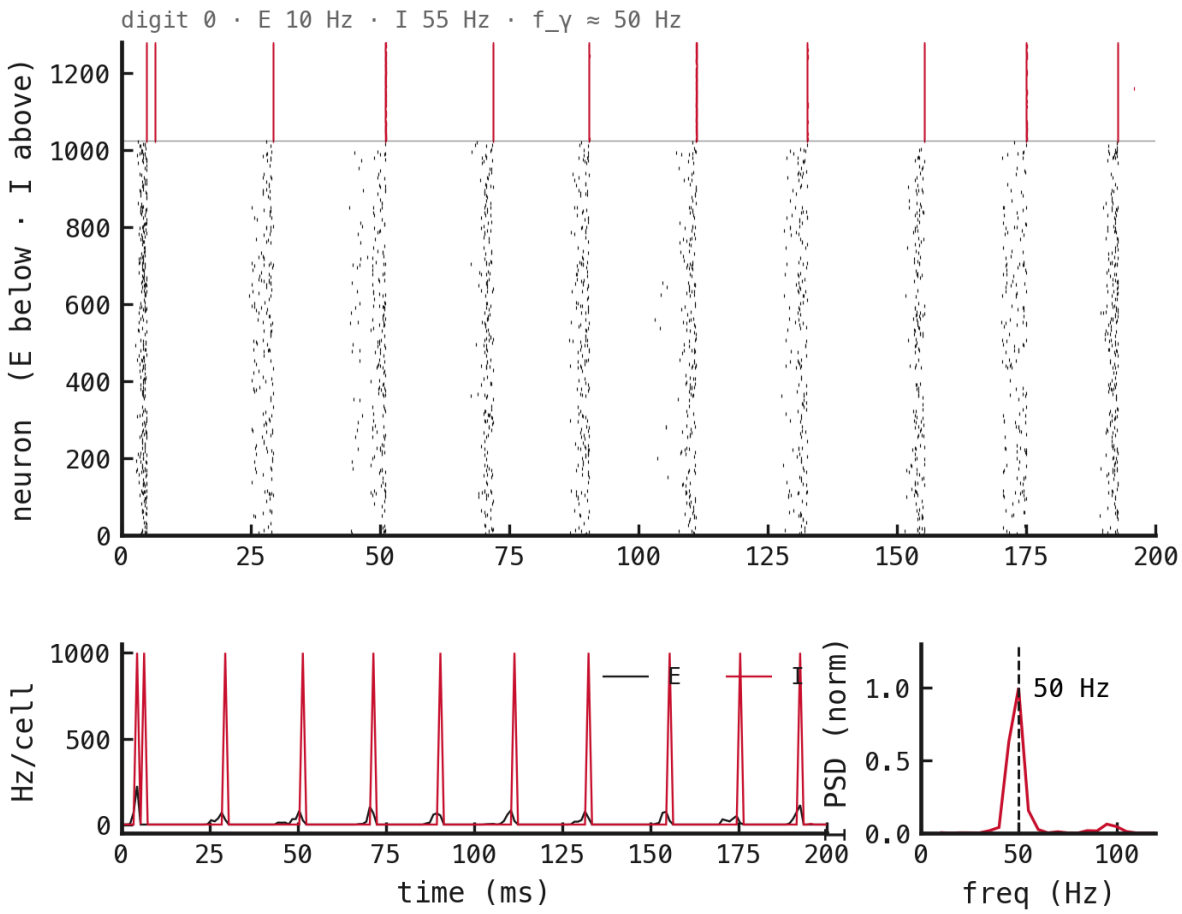

Figure 17: PING, spike budget = 2.

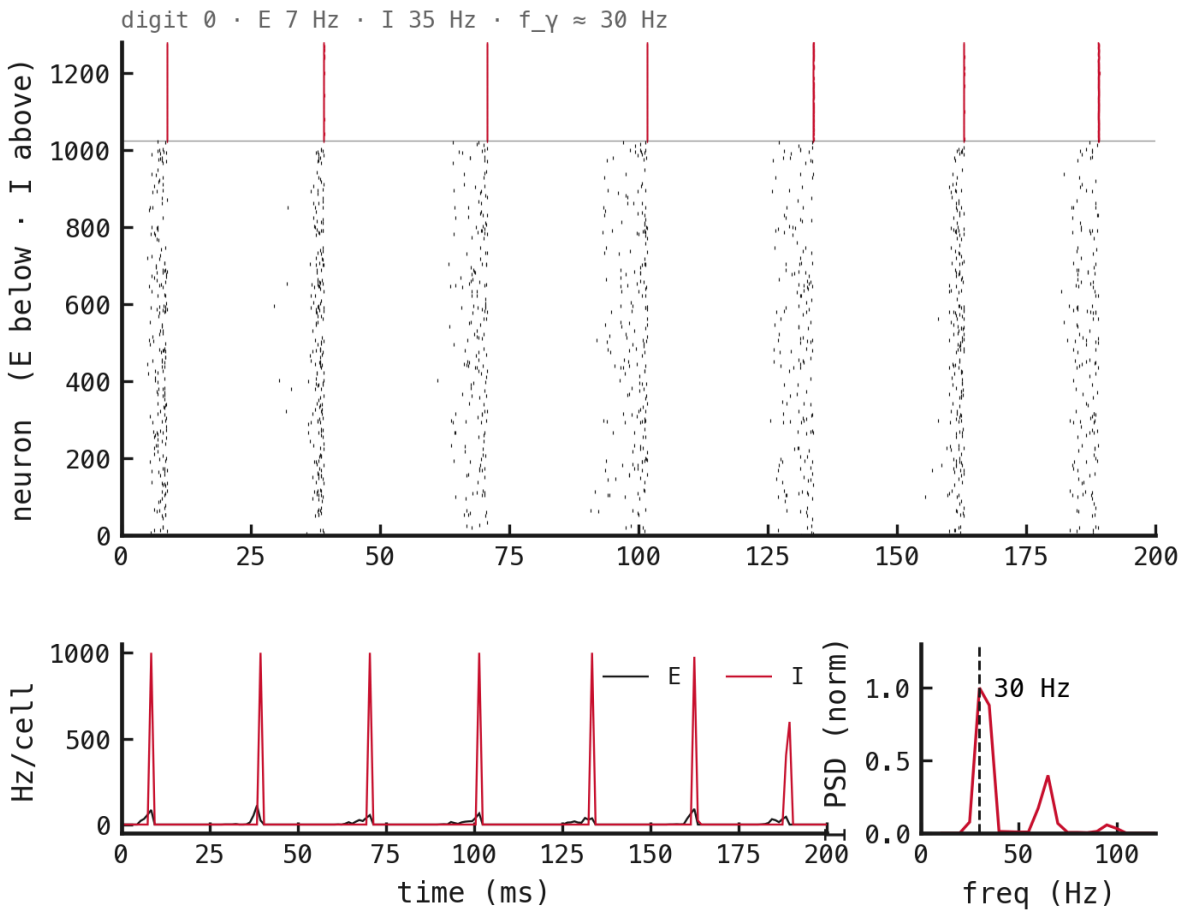

Figure 18: PING, spike budget = 1.

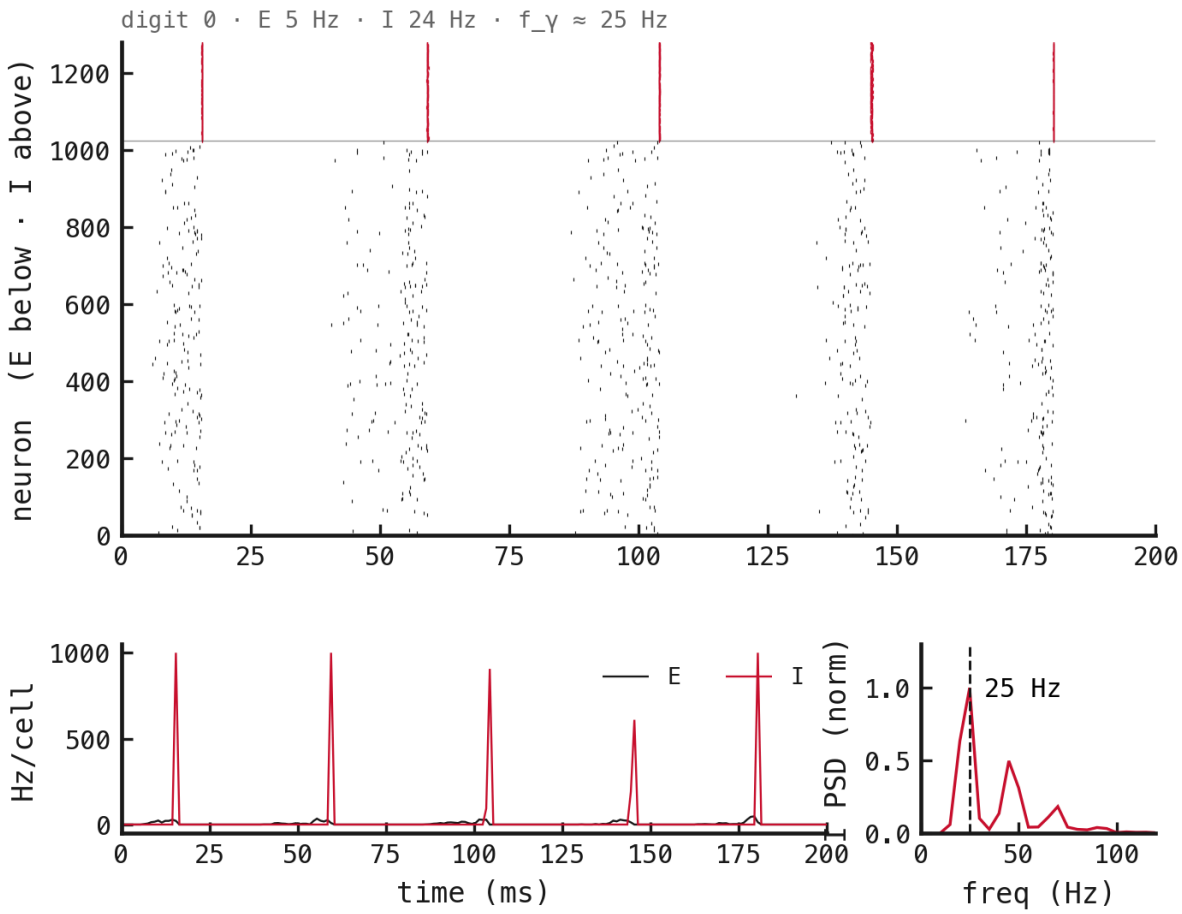

Figure 19: PING, spike budget = 0.5.

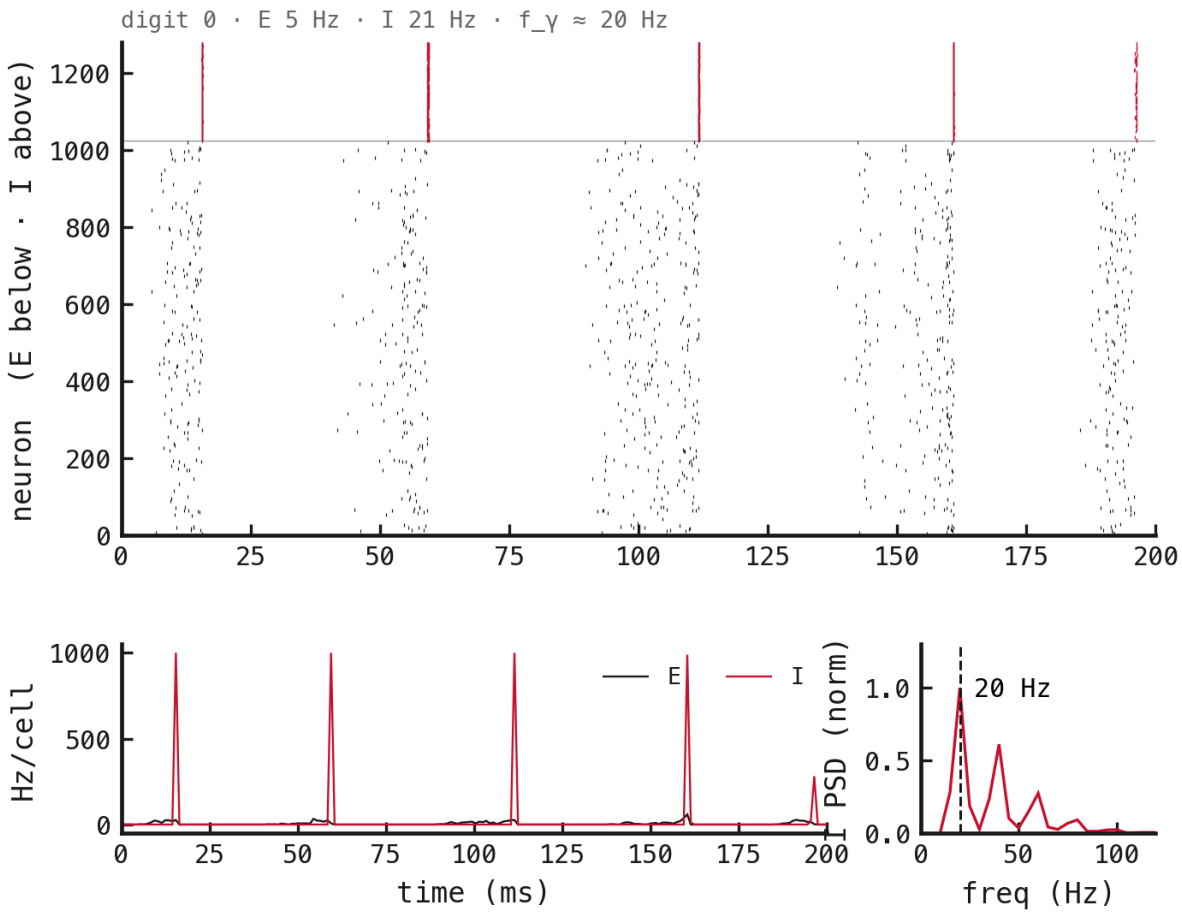

Figure 20: PING, spike budget = 0.2.

A.3 τ_GABA ladder

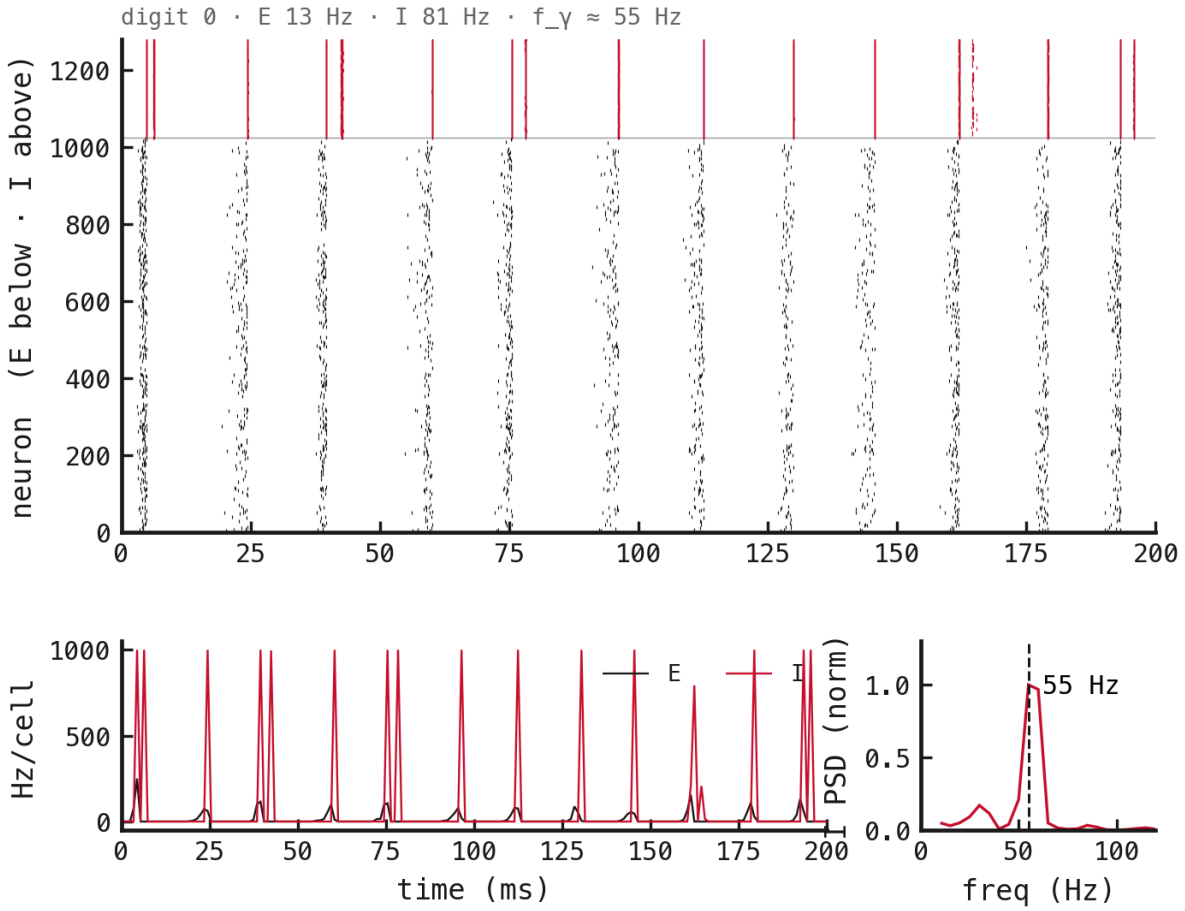

Figure 21: PING, τ_GABA = 4.5 ms.

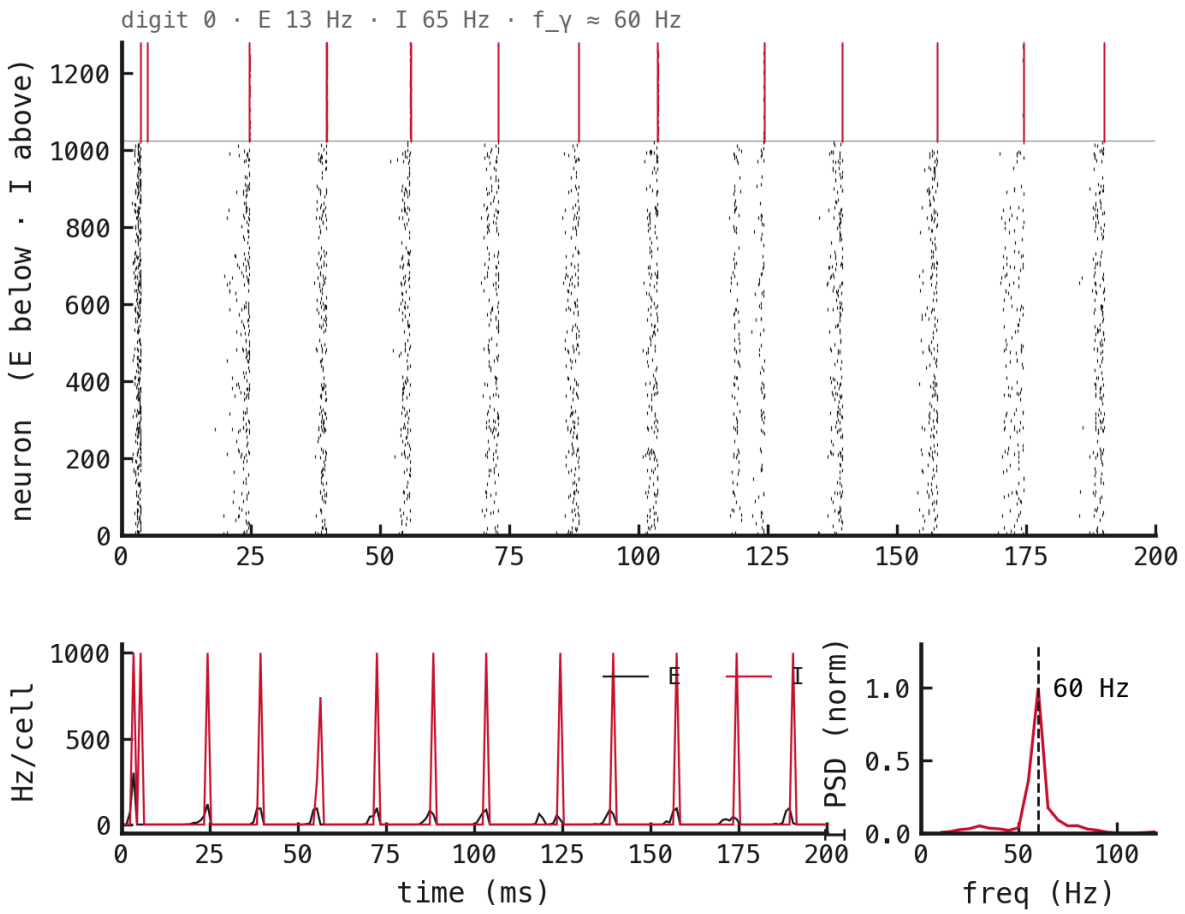

Figure 22: PING, τ_GABA = 6 ms.

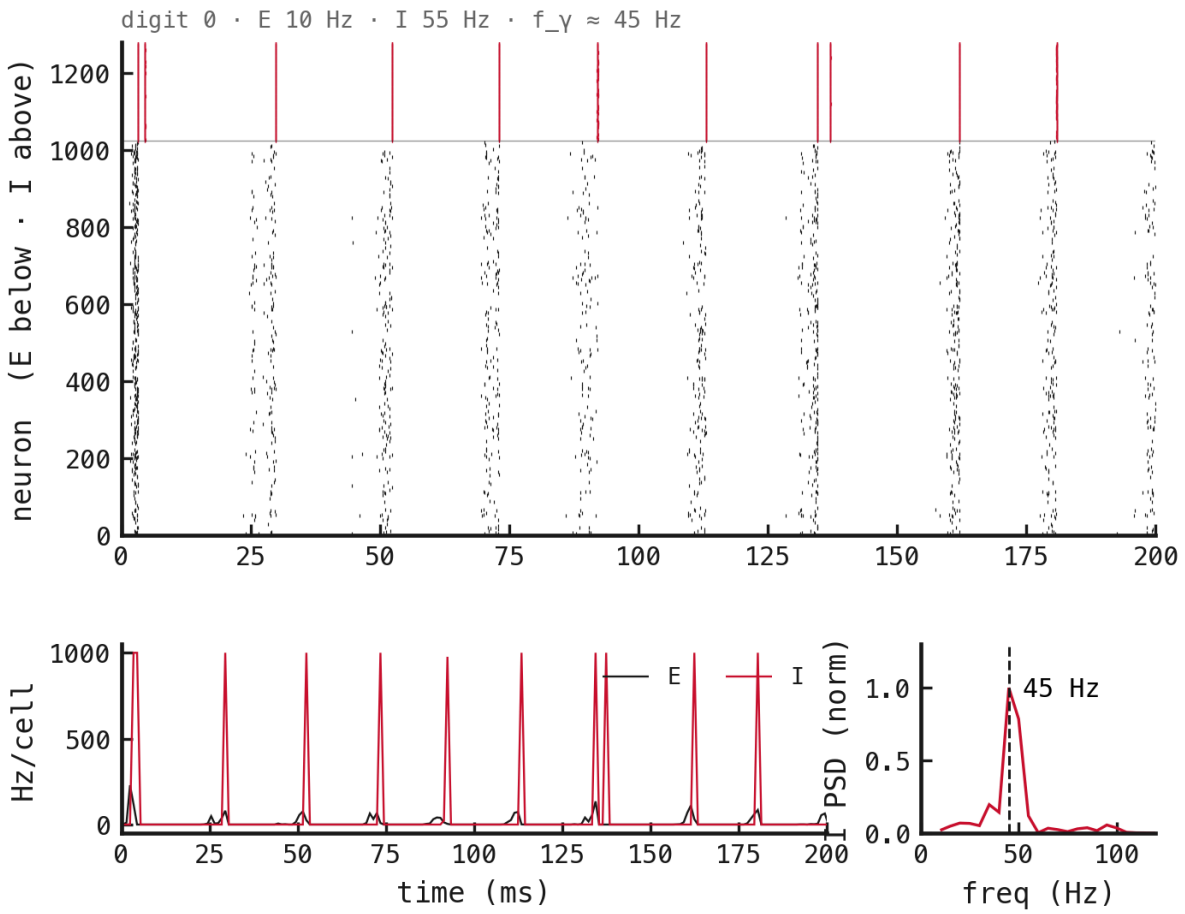

Figure 23: PING, τ_GABA = 9 ms.

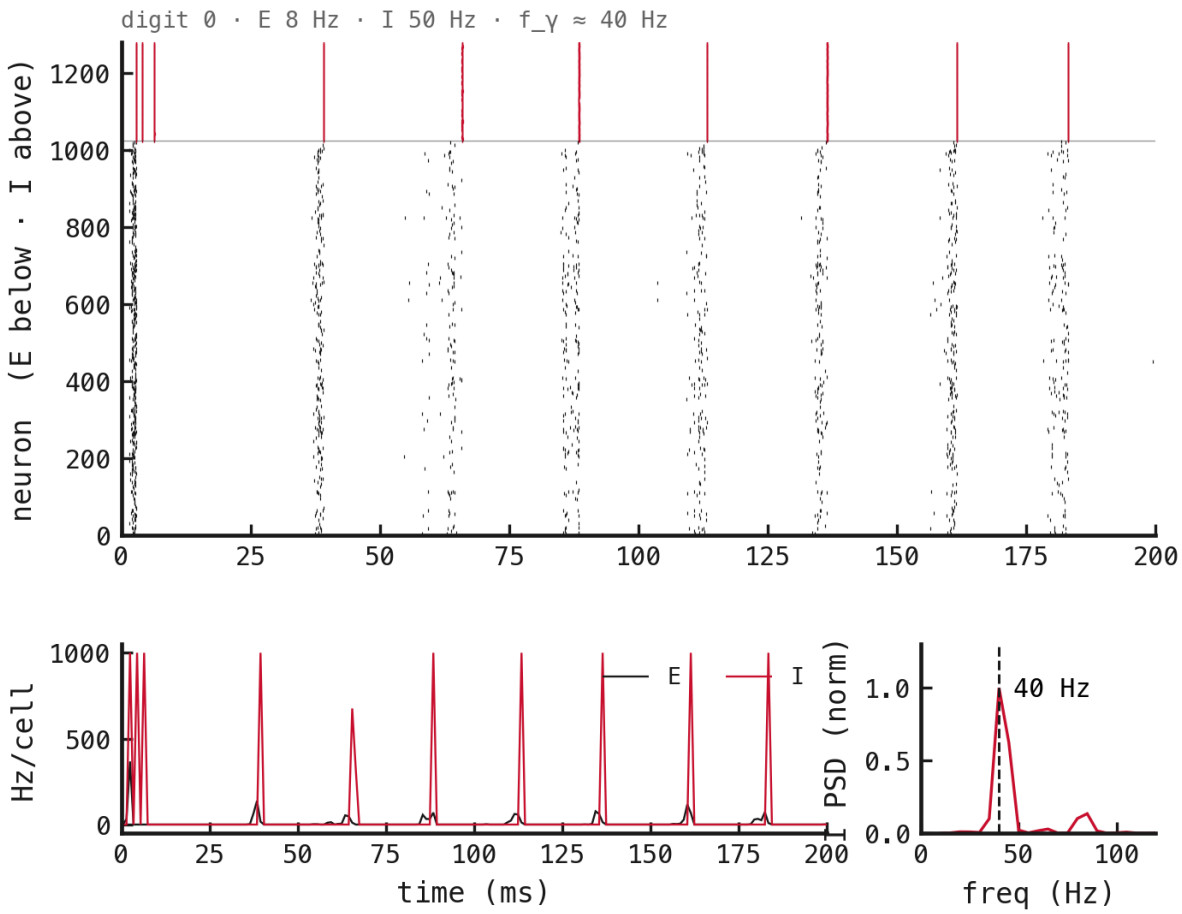

Figure 24: PING, τ_GABA = 12 ms.

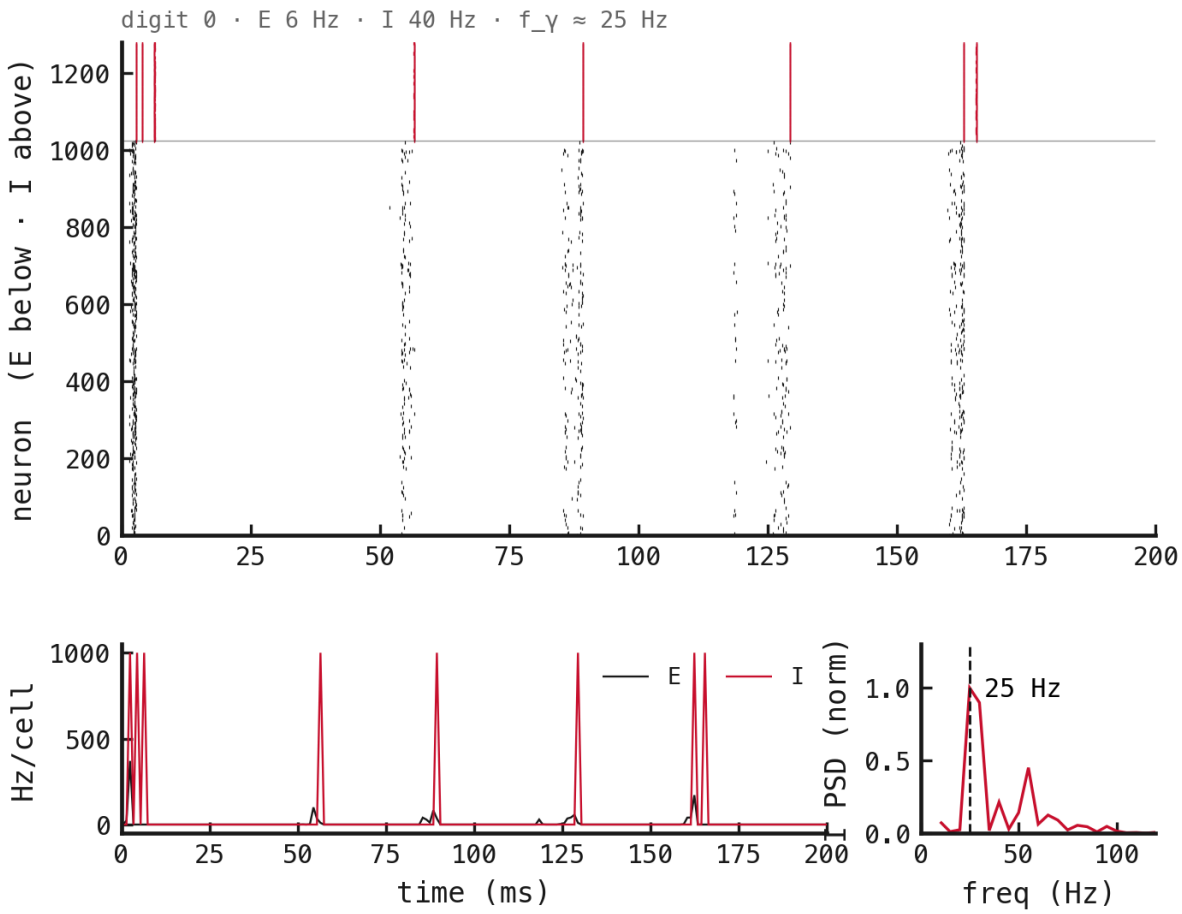

Figure 25: PING, τ_GABA = 18 ms.

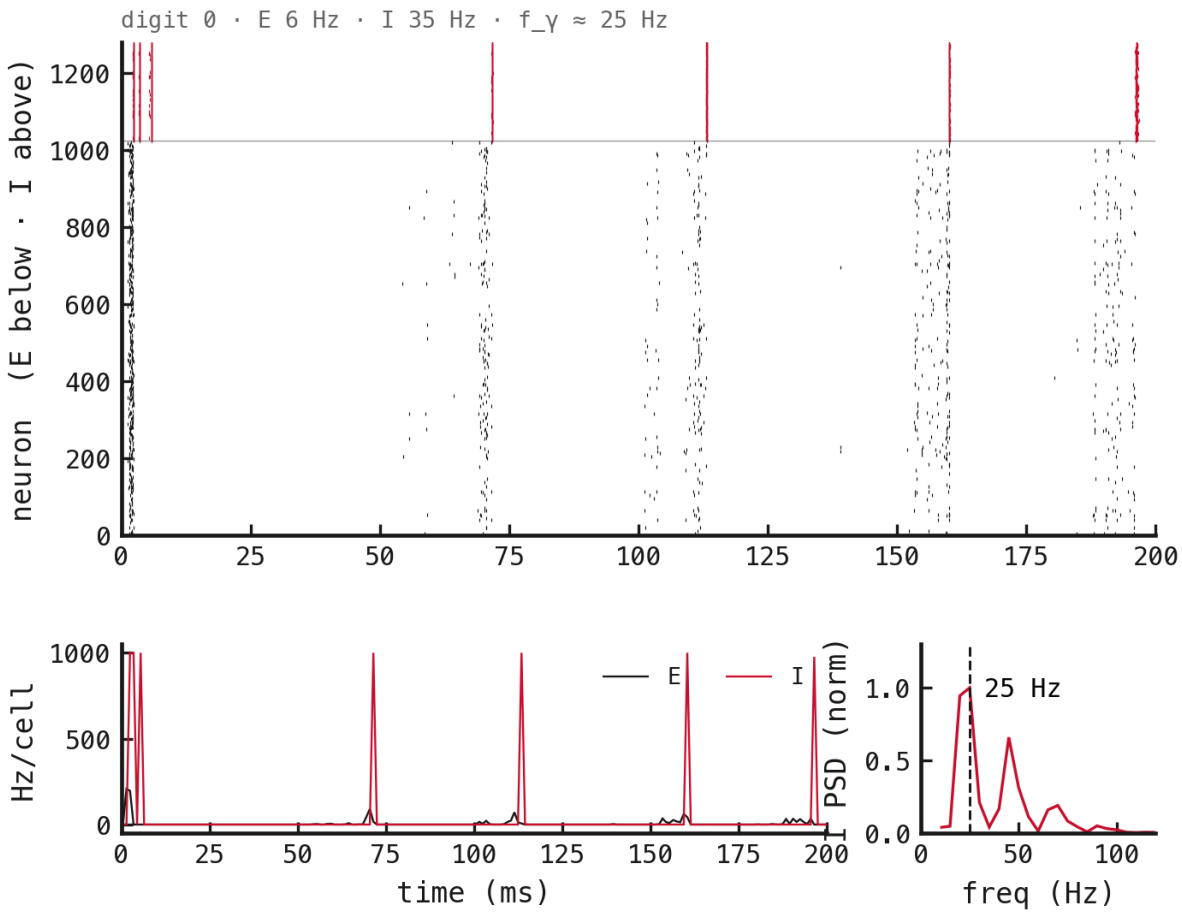

Figure 26: PING, τ_GABA = 27 ms.

A.4 Δt sweep

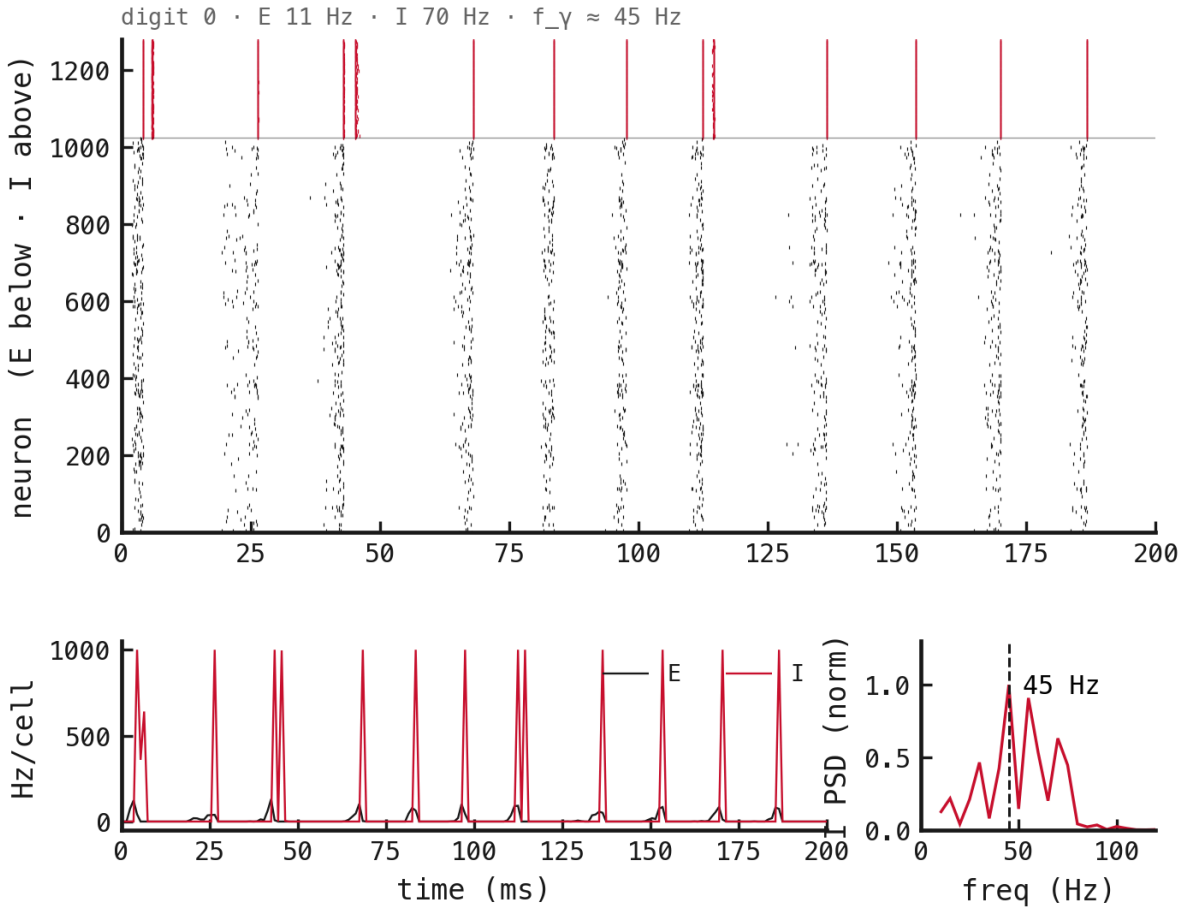

Figure 27: PING, $\Delta t = 0.05$ ms.

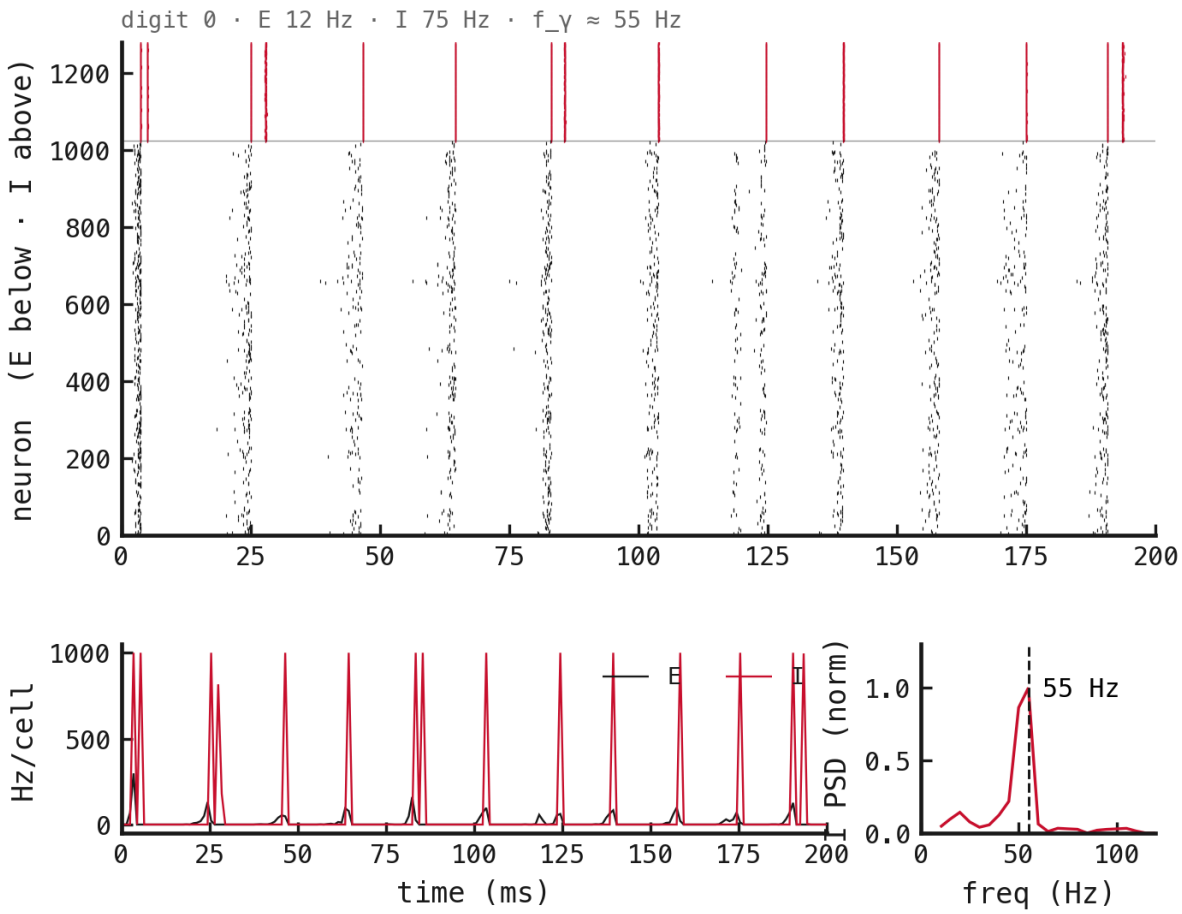

Figure 28: PING, $\Delta t = 0.1$ ms.

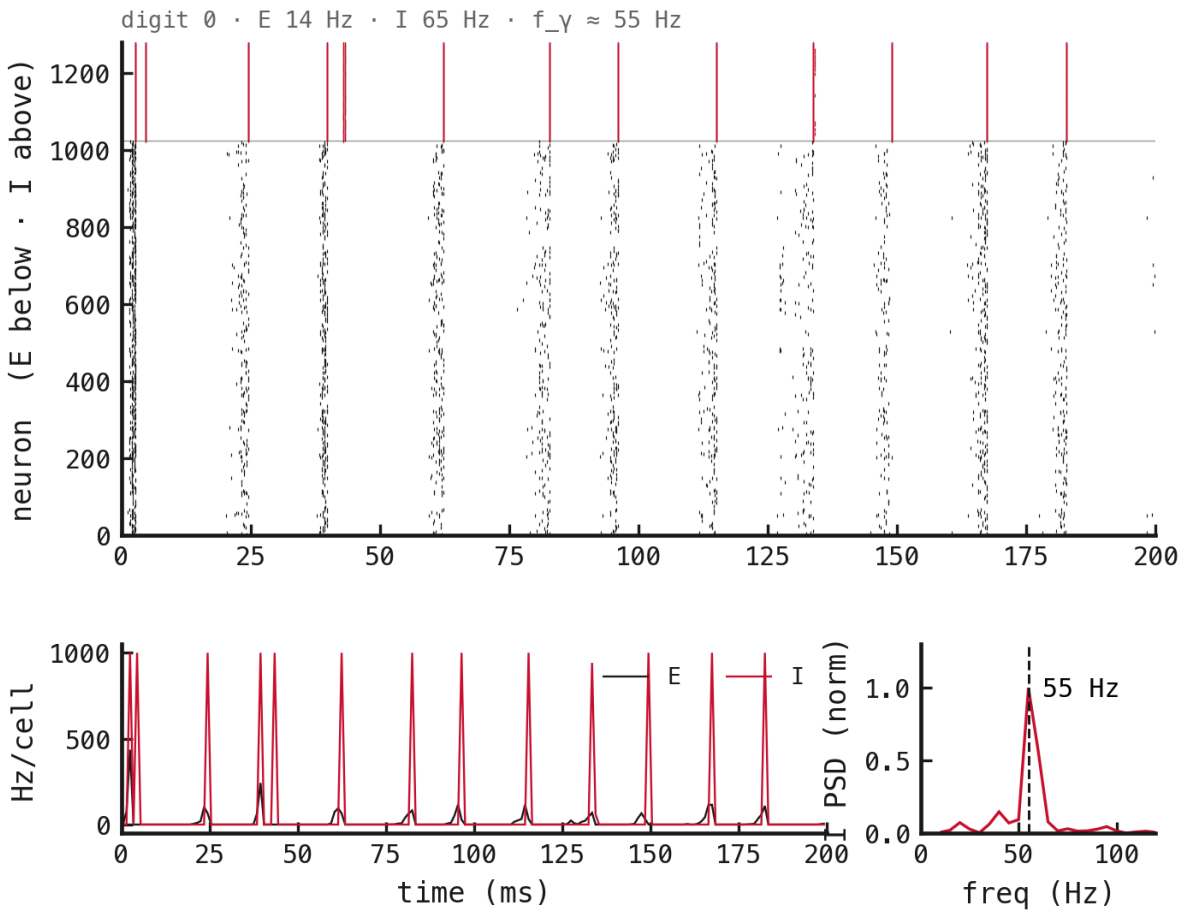

Figure 29: PING, $\Delta t = 0.25$ ms.

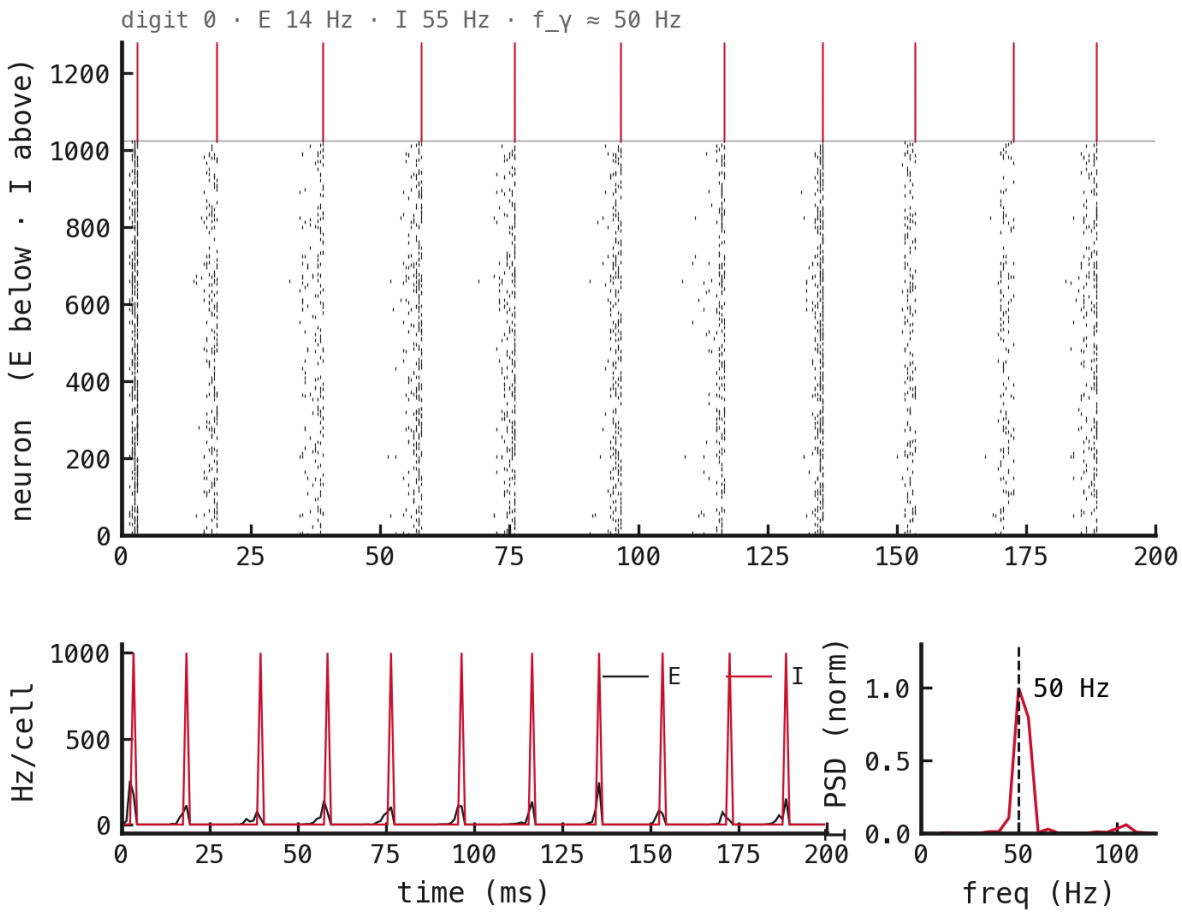

Figure 30: PING, $\Delta t = 0.5$ ms.

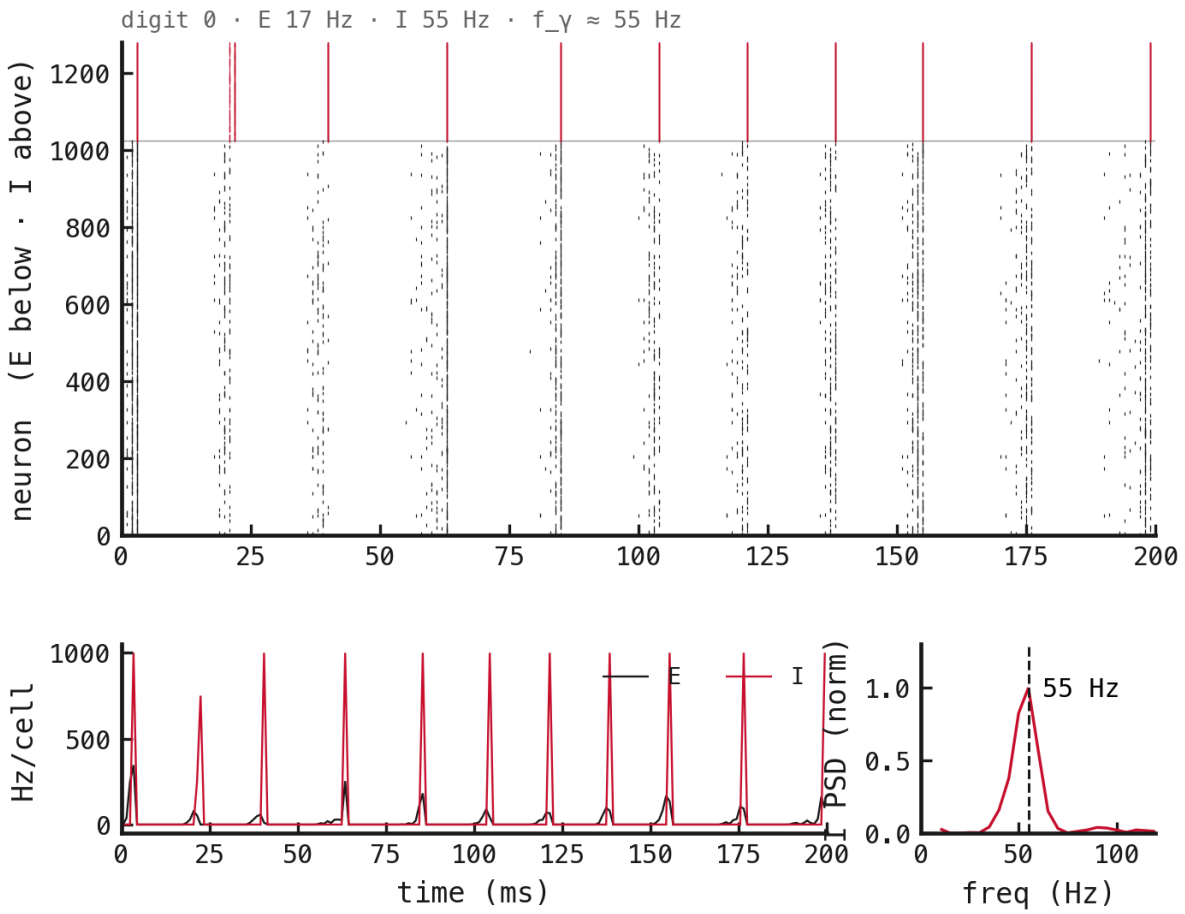

Figure 31: PING, $\Delta t = 1$ ms.

A.5 Init variants

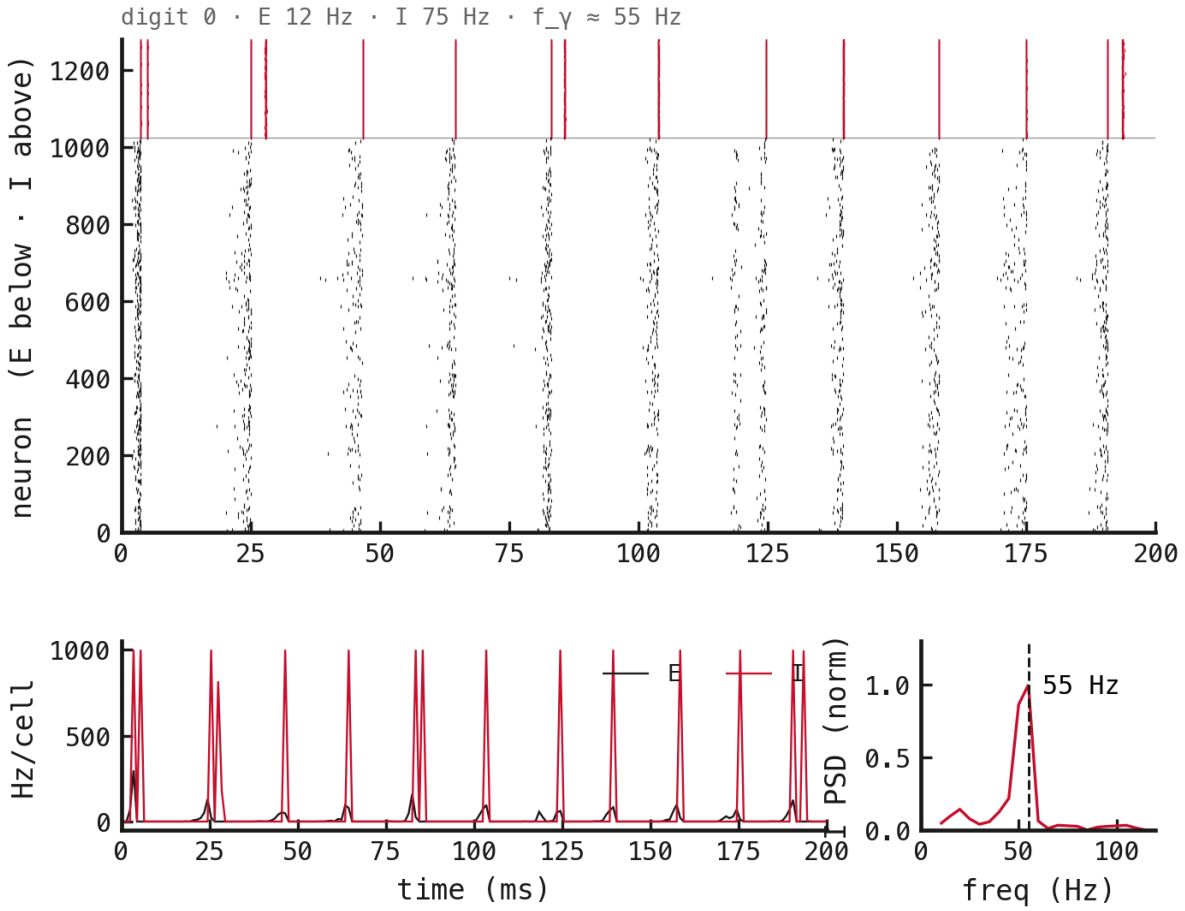

Figure 32: PING, frozen loop (control).

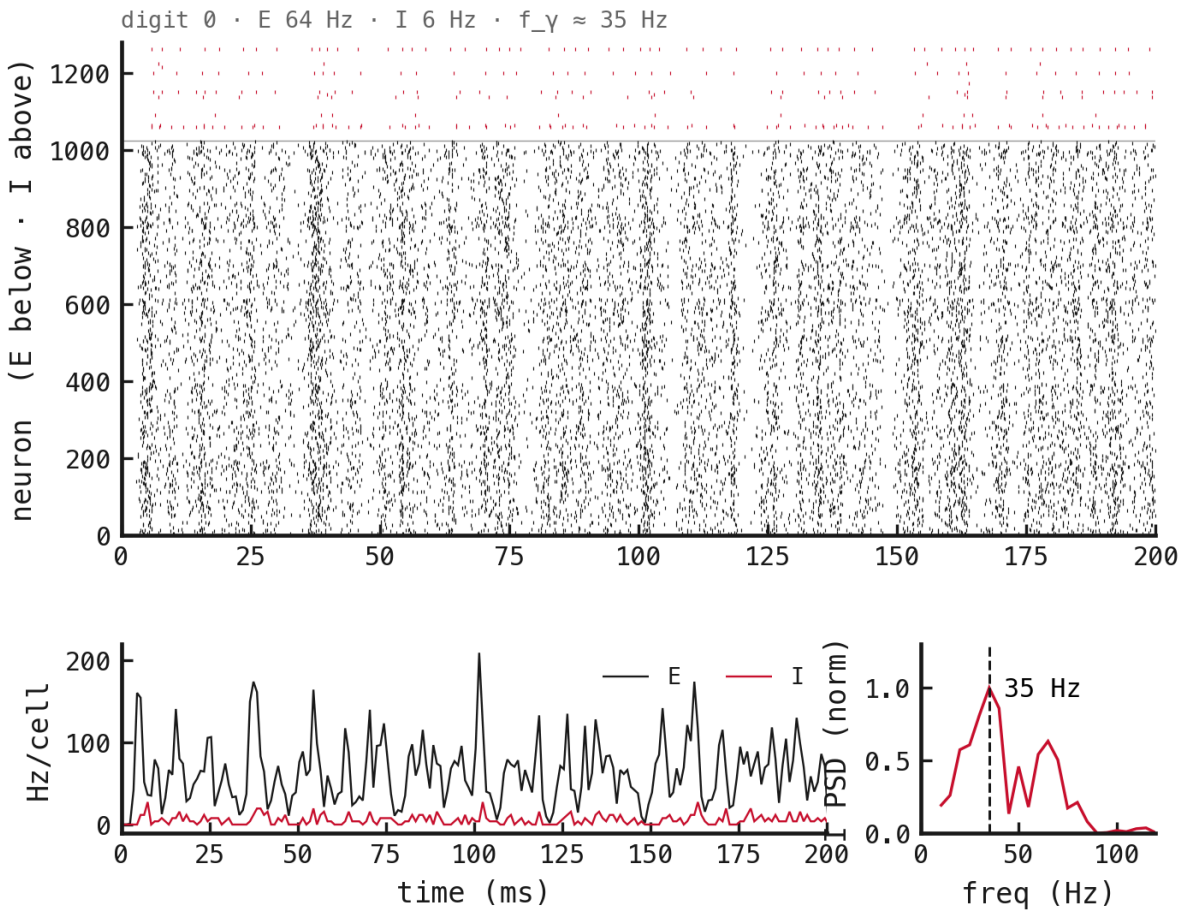

Figure 33: PING, trainable loop, PING init.

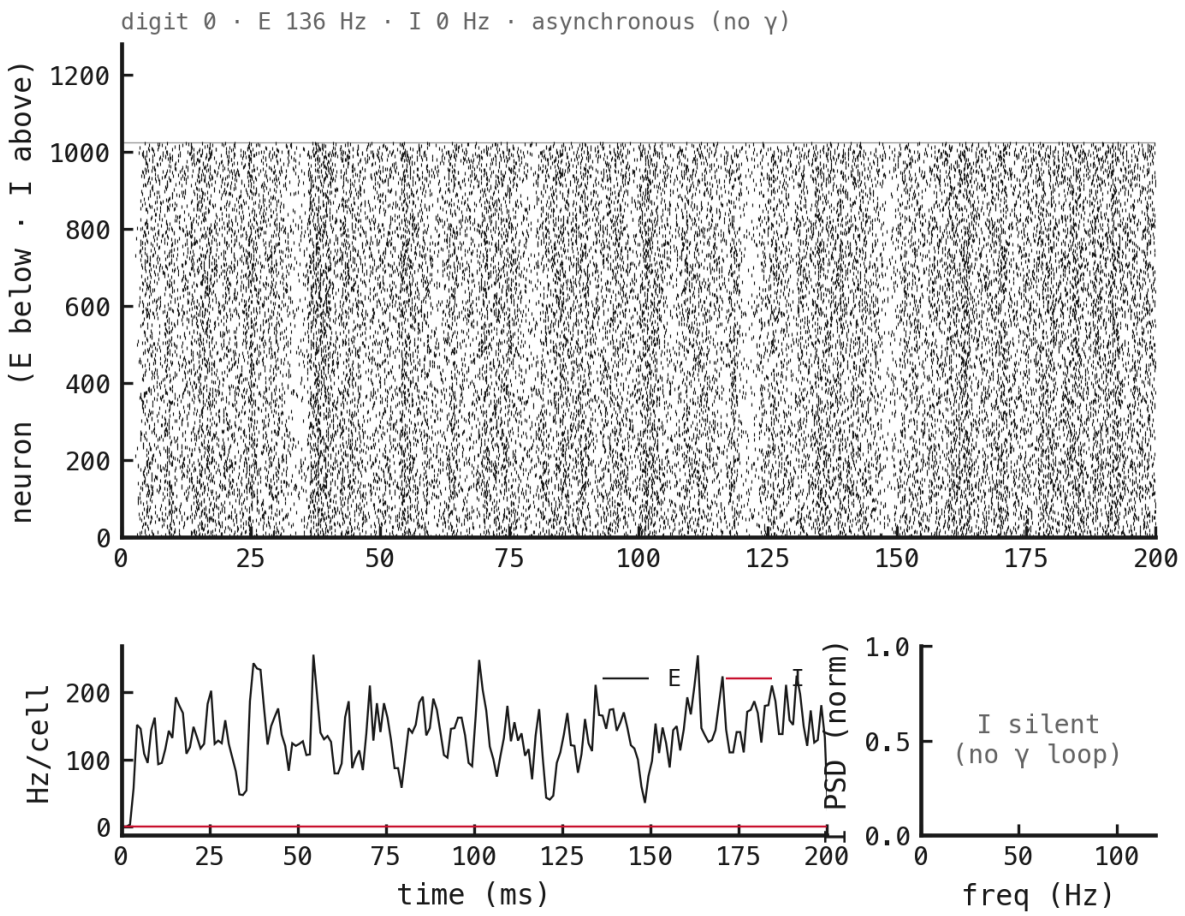

Figure 34: PING, trainable loop, zero init.

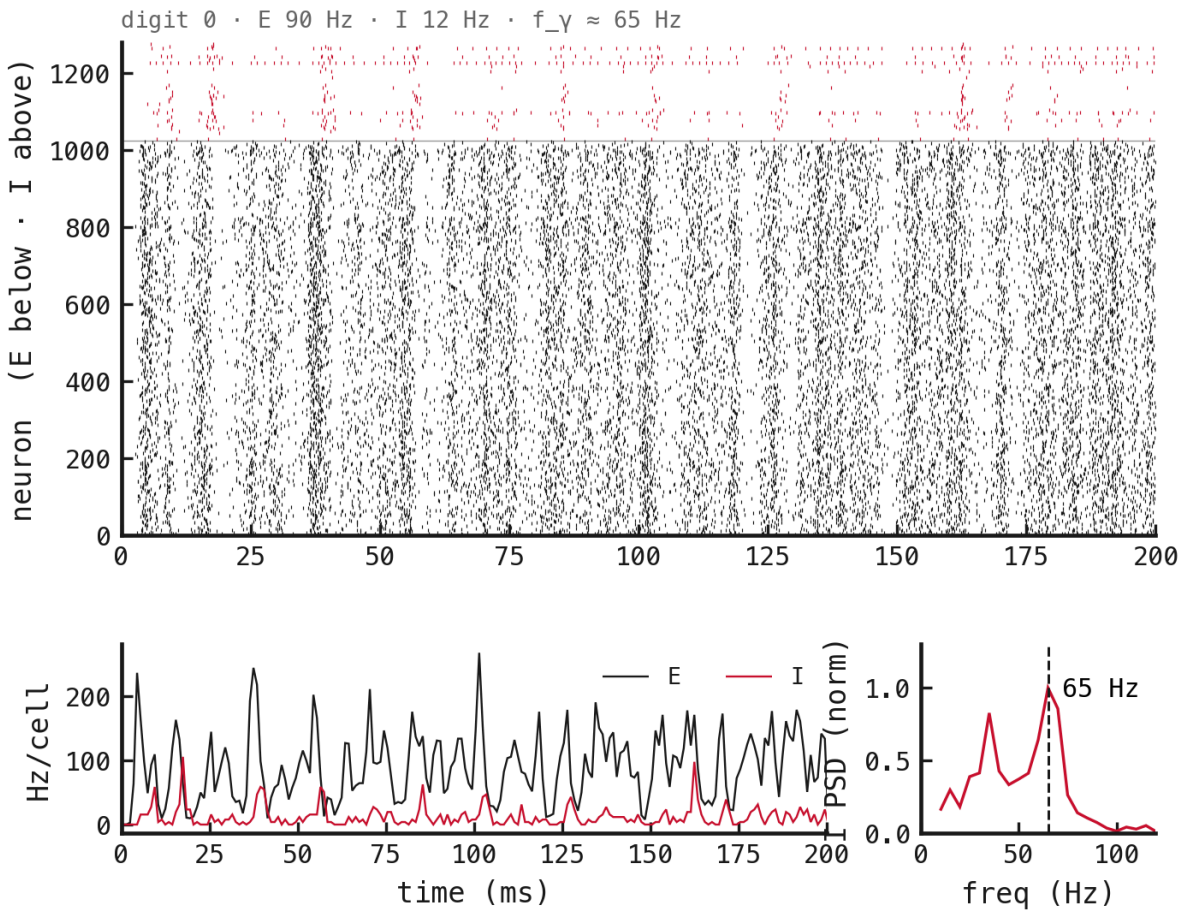

Figure 35: PING, trainable loop, small init.